UNIVERSITY OF SCIENCE
FACULTY OF INFORMATION TECHNOLOGY



OBJECT - ORIENTED PROGRAMMING

ROAD CROSSING GAME'S PROJECT

Lecturer: **Dr. Trương Toàn Thịnh**

Class: **25C11**
Group: **04**

25127040 - Nguyễn Bá Duy
25127212 - Trịnh Trần Hương Mai
25127367 - Nguyễn Duy Khang
25127431 - Trần Ngọc Nghĩa
25127559 - Thái Mạc Tường Vi

Ho Chi Minh city, 2026

I. TABLE OF CONTENTS

I. TABLE OF CONTENTS..... 2
II. GROUP INFORMATION AND WORK ASSIGNMENT..... 4
1. Group information..... 4
2. Work assignments..... 4
III. PROJECT OVERVIEW..... 7
1. Introduction..... 7
2. Features summary..... 7
3. Tools & libraries used..... 9
3.1. Development tools & infrastructure..... 9
3.2. Extra libraries: Simple and fast multimedia library (SFML 3.1.0)..... 9
4. Game resource footprint & memory estimation..... 11
4.1. Disk space footprint (ROM)..... 11
4.2. Runtime memory estimation (RAM & VRAM)..... 11
4.3. Measurement and profiling methodology..... 12
IV. SYSTEM STRUCTURE & ORGANIZATION..... 13
1. Game flowchart..... 13
2. Class diagram (UML)..... 14
3. Source code organization..... 15
4. Error handling system..... 16
4.1. Objective..... 16
4.2. System design..... 16
4.3. Exception classification & types..... 16
4.4. Exception hierarchy tree..... 17
4.5. Three-tier exception handling mechanism..... 17
V. CORE PROCESSING TECHNIQUES..... 18
1. Software architecture & programming paradigms..... 18
1.1. Object-oriented programming (OOP)..... 18
1.2. Design patterns..... 23
1.3. Advanced memory management (smart pointers)..... 30
1.4. Data structures & STL utilization..... 31
2. Game engine & runtime infrastructure..... 33
2.1. Game loop & delta time..... 33
2.2. Window management and adaptive resolution handling..... 36
3. Real-time game processing & mechanics..... 39
3.1. Input processing..... 39
3.2. Collision processing..... 41

3.3. Movement & traffic logic (steering behaviors).....49
3.4. Camera tracking & auto-scroll system.....54
3.5. Entity animation & sprite sheet system.....59
4. Environment & terrain systems.....67
4.1. Core architecture.....67
4.2. Terrain specifications.....67
5. Audio & sound system.....69
5.1. Game audio classification.....69
5.2. Optimization techniques.....69
6. Data persistence.....71
6.1. Save/load game processing.....71
6.2. Binary file serialization & deserialization.....72
6.3. Binary file encryption.....72
VI. CONCLUSION.....74
1. Project achievements & functional delivery.....74
2. Software architecture & OOP application.....74
3. Stability, memory management & profiling.....75
4. Technical takeaways & team growth.....75
VII. LIMITATIONS & FUTURE ENHANCEMENTS.....75
1. Current system limitations.....75
2. Future enhancements roadmap.....76
VIII. REFERENCES.....77
IX. ACKNOWLEDGMENTS.....78

II. GROUP INFORMATION AND WORK ASSIGNMENT

1. Group information

No.	Student code	Full name	Email	Position
1	25127040	Nguyễn Bá Duy	nbduy2506@clc.fitus.edu.vn	Leader
2	25127212	Trịnh Trần Hương Mai	tthmai2536@clc.fitus.edu.vn	Member
3	25127367	Nguyễn Duy Khang	ndkhang2526@clc.fitus.edu.vn	Member
4	25127431	Trần Ngọc Nghĩa	tnnghia2529@clc.fitus.edu.vn	Member
5	25127559	Thái Mạc Tường Vi	tmtvi2535@clc.fitus.edu.vn	Secretary

2. Work assignments

Student code	Full name	Assignments	
25127040	Nguyễn Bá Duy	Write report	Project overview
			Game flowchart
			Source code organization
			Object-oriented programming (OOP)
			Audio & sound system
			Conclusion
			Limitations & future enhancements
		Base Program	State & ShareContext classes hierarchy
			AudioSystem class
		Features	Audio & SFX System
			Score & Highscore System
			In-Game Skin Shop & Character Selector
		Other	Tracking progress

25127212	Trịnh Trần Hương Mai	Write report	Class diagram (UML)
			Data structures & STL utilization
			Game loop & delta time
			Environment & terrain systems
		Base Program	Map class
			Lane class hierarchy
		Features	Infinite lane and object generation
			Moving Platforms (in Wood and River)
		Other	Draw background, buttons graphics
25127367	Nguyễn Duy Khang	Write report	Design patterns
			Input processing
			Collision processing
			Entity animation & sprite sheet system
		Base Program	People class
		Features	Entities Animation
		Other	Draw Bird, Rabbit, Car, Truck, People sprite sheets; Traffic graphic
25127431	Trần Ngọc Nghĩa	Write report	Game resource footprint & memory estimation
			Error handling system
			Advanced memory management (smart pointers)
			Window management and adaptive resolution handling
			Collision processing
			Data persistence
		Base Program	Game, People classes
		Features	Save/Load game
			Logic increases game difficulty
			Coin collection & economy
		Other	Draw Rock, Bush, Coin graphics

25127559	Thái Mạc Tường Vi	Write report	Introduction
			Class diagram (UML)
			Data structures & STL utilization
			Movement & traffic logic (steering behaviors)
			Camera tracking & auto-scroll system
		Base Program	DynamicObject & StaticObject classes hierarchy
			ResourceManager class
		Features	Camera tracking & Hardmode forced-scroll
			Steering behaviors (Vehicle)
		Other	Draw Lane graphics & sprite sheet

III. PROJECT OVERVIEW

1. Introduction

This report presents an overview to the detailed scenario and architecture of the Crossing game developed by Group 04. This is a console game developed in C++ using an object-oriented programming approach.

The ***Road Crossing*** game is inspired by the classic road-crossing genre, in which the player controls a character to cross various lanes with different obstacles. The objective is to guide the character as far as possible while avoiding obstacles to achieve the highest score.

2. Features summary

No.	Features	Description	Status
1	State Machine Management	Seamlessly transitions between 8 screens: Home, Playing, Pause, Settings, Ranking, Load, Save, and Game Over.	Completed
2	Infinite Map Spawning	Dynamically generates random lanes based on the player's forward movement and deallocates old lanes left behind.	Completed
3	Player Movement Control	Processes keyboard inputs (W/A/S/D or arrow keys) to move the player in a grid-based manner, matching the lane size.	Completed
4	Scrolling View Camera	The camera automatically scrolls vertically (Y-axis) as the player advances to expand the visible world.	Completed
5	Vehicles	Cars and trucks move along road lanes and are capable of automatically braking to maintain a safe distance from vehicles ahead.	Completed
6	Animal	Birds and rabbits move across grass lanes and automatically stop when getting too close to another animal ahead.	Completed
7	Traffic Light System	Traffic lights cycle through Green → Yellow → Red; vehicles detect the light status to slow down, stop, or accelerate.	Completed

8	Collision Detection	Handles precise collision checks (using a 0.9 hitbox ratio) between the player and vehicles/animals, or when the player falls out of the camera's bottom view.	Completed
9	Rock and grass	Static objects on the map spawn randomly on the lanes; they are not fatal to players.	Completed
10	Score & Highscore System	Calculates scores based on the maximum lane distance reached; automatically sorts and saves the Top 5 Highscores to a binary file highscore.bin.	Completed
11	Configurations (Settings)	UI elements to increase or decrease Music and SFX volume levels ranging from 0% to 100%.	Completed
12	Audio & SFX System	Plays background music during gameplay alongside car engine sounds, honks, animal noises, and collision alerts.	Completed
13	Save & Load Game Logic	Accurately serializes and saves the player's position and map lane data to allow resuming the game later.	Completed
14	Rivers & Logs Terrain	Waterway lanes featuring floating river currents, randomly spawning logs and turtles for the player to hop on.	Completed
15	Entity Animations	Visual squash-and-stretch animation effects for the player jumping, or animations for getting crushed upon collision.	Completed
16	Coin Collection & Economy	Randomly spawns coins on lanes (roads, grass, floating logs). Collect to build in-game currency used for unlocking items in Shop.	Completed
17	In-Game Skin Shop & Character Selector	Preview and purchase 3 unique characters (Person, Robot, Usagi) with coins. Automatically saves and equips chosen skin directly into gameplay.	Completed
18	Hardmode: Forced-Scroll	Screen auto-scrolls forward with speed increasing by level. Features a fatal trailing fire wall that forces continuous forward movement.	Completed

3. Tools & libraries used

3.1. Development tools & infrastructure

- **Integrated development environments (IDEs):**
  - Visual Studio 2022: Used as the primary IDE on Windows systems for heavy-duty compilation, native MSVC debugging, and structured solution management.
  - VS Code: Utilized as a lightweight text editor for rapid code modification, script management, and MinGW/GCC cross-compilation configurations.
  - CMake: Used as the project's meta-build system to configure compiler flags, automate global source/header file scanning (file(GLOB_RECURSE)), and resolve external SFML 3.1.0 library dynamic linkages for both MSVC (Visual Studio) and MinGW/GCC (VS Code) environments. It also handles deployment automation via custom post-build commands to automatically copy runtime assets, binary highscore matrices, and dynamic-link libraries (.dll) directly into the target build output directory.
- **Version control & collaboration:**
  - Git: Employed for local source code version control, staging changes, and tracking modification history through branches.
  - GitHub: Used as the remote cloud repository host to collaborate, merge code baselines, track project milestones, and backup incremental source code versions.
- **Documentation & project management:**
  - Google Drive (Docs, Sheets): Utilized to store asset spreadsheets, collaborate in real-time on report drafts, write development logs, and organize weekly submission planning.
  - Draw.io: Applied to architectural software design diagrams, including the State Machine transition flowcharts, class hierarchies, and system layout mappings.

3.2. Extra libraries: Simple and fast multimedia library (SFML 3.1.0)

- `SFML::Window`: Responsible for spawning the main OS window context (`sf::RenderWindow`), managing display resolutions, limiting frame rates (`window.setFramerateLimit`), and capturing interactive user window closing, keyboard, or mouse polling events.
- `SFML::Graphics`: Powers the hardware-accelerated 2D rendering engine. It handles image loading and runtime texture rendering (`sf::Texture`, `sf::Sprite`), text

rendering for UI text systems (`sf::Text`, `sf::Font`), camera view manipulation (`sf::View`), and collision geometric matrices (`sf::FloatRect`).

- `SFML::System`: Provides high-resolution timeline tracking via `sf::Clock`. This class measures the exact time elapsed between game loops as delta time (`dt`), which ensures consistent entity speeds and frame-independent simulation across varying CPUs.
- `SFML::Audio`: Integrated to manage spatial audio streaming, play sound effect channels, and load audio buffers (`sf::SoundBuffer`) from disk into internal memory matrices.

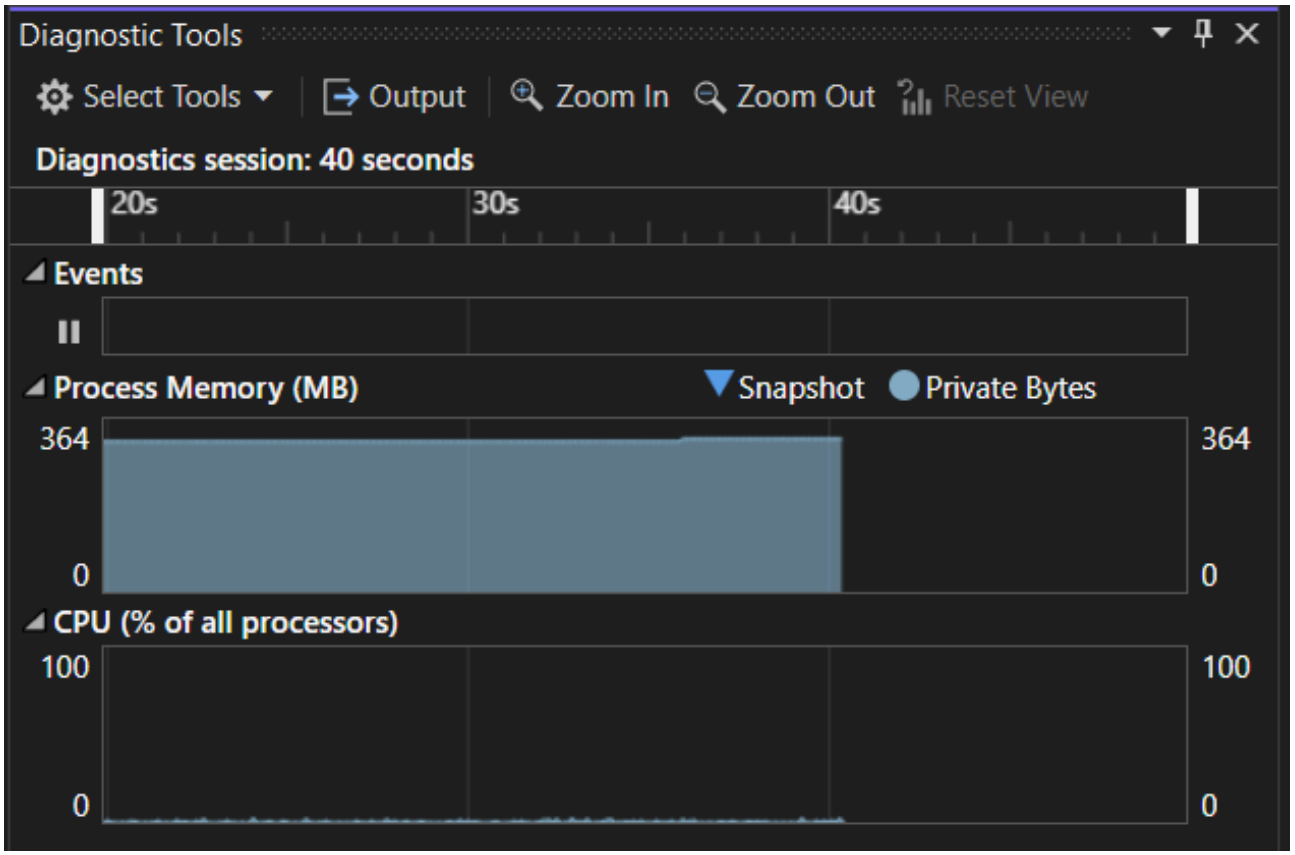
4. Game resource footprint & memory estimation

4.1. Disk space footprint (ROM)

The game is engineered to maintain a minimal storage footprint:

- **Compiled executable** (`.exe`): ≈ 2~5 MB (Optimized C++ release binary).
- **Asset media** (`assets/`): ≈ 15~25 MB (Compressed 2D PNG sprites, TTF fonts, OGG/WAV audio).
- **Binary save files** (`data/`): < 10 KB per slot (Lightweight encrypted binary format vs. heavy JSON/XML formats).

4.2. Runtime memory estimation (RAM & VRAM)



Measured in Visual Studio 2022 Diagnostic Tools, the total process memory footprint is strictly controlled at ~370 MB, categorized into:

- **GPU Driver & OpenGL runtime:** ≈150~200 MB. Allocated by the OS graphics driver upon window creation to manage the hardware-accelerated OpenGL context, swapchain (double buffering), and display framebuffers.

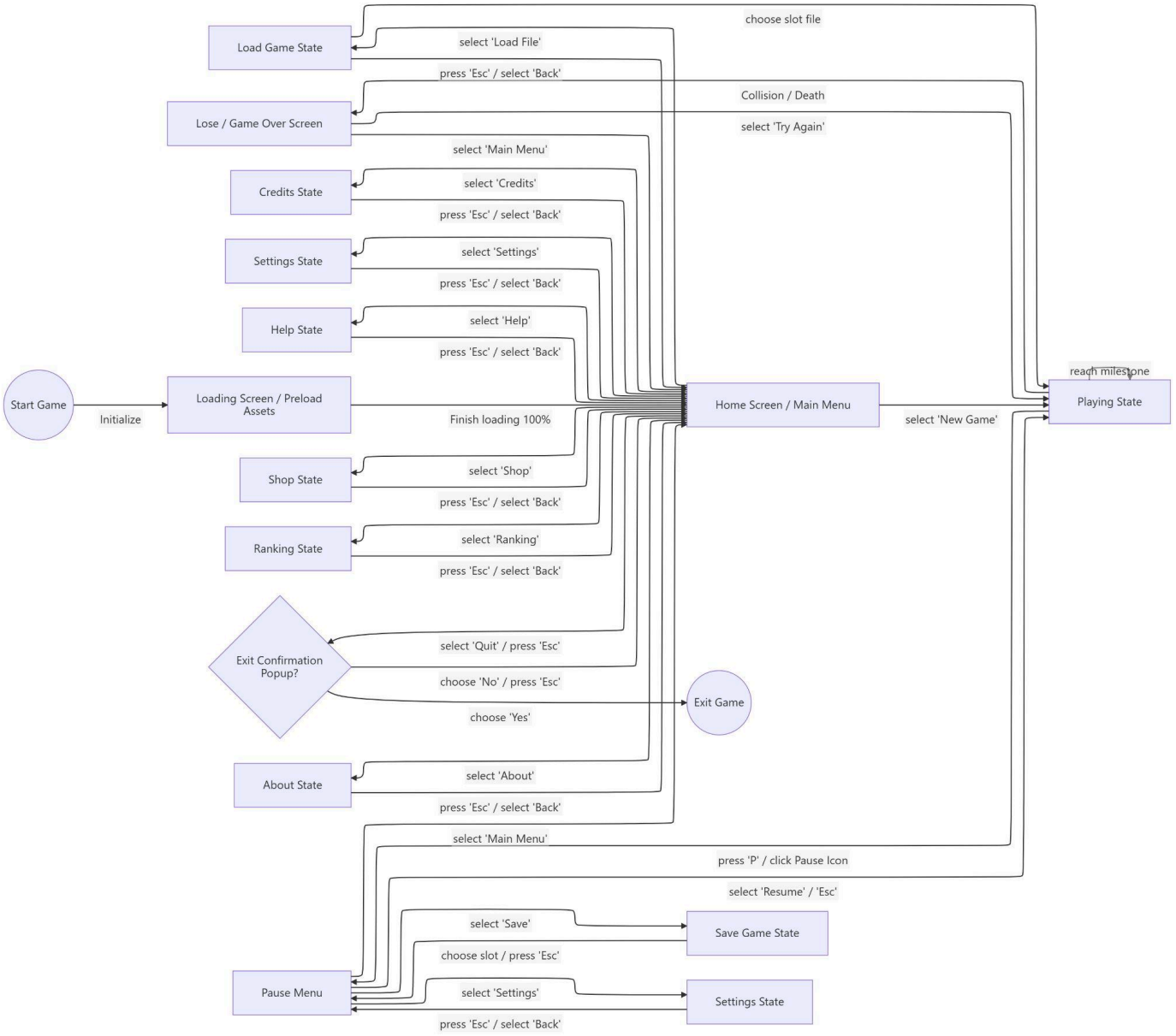
- **VRAM utilization (textures)**: ≈60~100 MB. Managed by `ResourceManager` using the **Flyweight pattern** (textures are loaded once into GPU VRAM and shared across all active sprites).
- **Heap RAM utilization (Game objects & logic)**: ≈15~30 MB. Dynamic entities are managed via C++11 smart pointers (`std::unique_ptr`), ensuring zero memory leaks during entity creation and destruction.

4.3. Measurement and profiling methodology

- Measurement tools: Memory usage was profiled using Visual Studio Diagnostic Tools and CRT Memory Leak Detection (`_CrtDumpMemoryLeaks`).
- Optimization Strategies:
    1. Texture caching avoids duplicate asset loading.
    2. Binary serialization (`Serialization::write`) eliminates memory allocation overhead associated with string parsing.
    3. Culling out-of-bounds map lanes (`popLane()`) frees memory for off-screen entities.

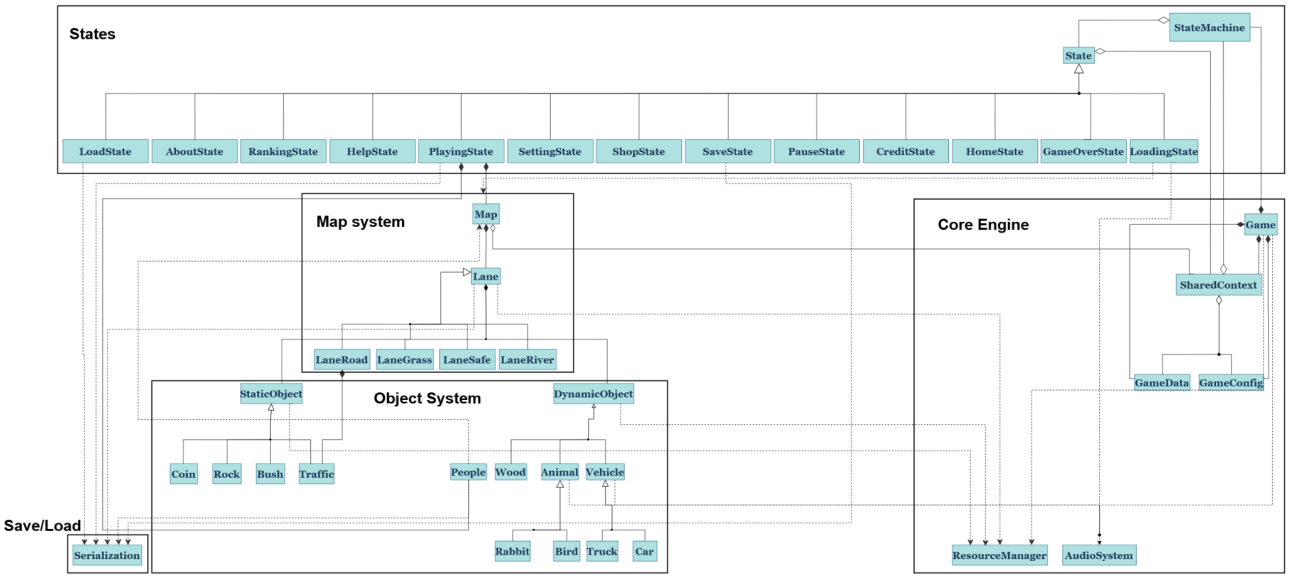
IV. SYSTEM STRUCTURE & ORGANIZATION

1. Game flowchart



2. Class diagram (UML)

The team's current UML overview:



There are 5 main system groups of the game:

- **Core Engine:** These are core classes responsible for running the game loop, managing data, audio system, and switching between states.
- **Game State:** This group handles the different screens and states of the game using the State design pattern.
- **Map System:** This group is responsible for generating and managing the world. Types of lanes are separate objects that are inherited by `Lane`.
- **Object System:** This group is all about the objects, including character, vehicle, animal, and traffic. It also plays a role in processing the interactions between objects.
- **Save/Load:** Serialization is used to implement the Save/Load Game feature.

Download PDF file to see the complete UML diagram: Here.

3. Source code organization

```
RoadCrossing/
├── .vscode/                    # Development environment configuration for VS Code
├── assets/                     # Directory containing original game resources
│   ├── audio/                  # Sound effects and background music
│   ├── fonts/                  # Display fonts
│   └── textures/               # Graphical assets, character/vehicle sprites
├── data/
│   └── saves/                  # Storage partition for game save files
├── include/                    # Directory containing all header files (.h)
│   ├── core/                   # Main game loop and State Machine management
│   ├── entities/               # Entity definitions
│   ├── states/                 # Screen state definitions
│   ├── utils/                  # Utility classes and constants
│   └── world/                  # Map structure components
├── libs/                       # External linked libraries
│   ├── SFML-3.1.0-...-mingw/   # Precompiled SFML library for VS Code
│   └── SFML-3.1.0-...-vc17/    # Precompiled SFML library for Visual Studio
├── src/                        # Directory containing all source implementation files
│   ├── core/
│   ├── entities/
│   ├── states/
│   ├── utils/
│   ├── world/
│   └── main.cpp                # Program entry point (Main function)
└── CMakeLists.txt
```

4. Error handling system

4.1. Objective

In game engine architecture, unhandled runtime exceptions lead to abrupt application crashes (`std::terminate`), resulting in poor user experience and potential save-data corruption. The primary objectives of our custom Error handling system are:

- **Fault tolerance & stability:** Prevent fatal crashes by intercepting low-level runtime errors (missing assets, corrupted binary data, invalid state transitions).
- **Graceful degradation:** Provide recovery paths for non-critical failures (e.g., resetting corrupted save files to default values instead of aborting the game).
- **Diagnostic logging:** Supply detailed exception messages (`e.what()`) to assist developers during debugging and quality assurance.

4.2. System design

The system is built around a centralized exception header (`include/utils/game_exceptions.h`). It extends the standard C++ exception framework (`std::exception`) to create a domain-specific exception hierarchy tailored for SFML game engines.

4.3. Exception classification & types

The framework categorizes errors into three functional domains:

- **ResourceException**: Thrown by `ResourceManager` when an asset (PNG texture, TTF font, WAV audio) fails to load from disk.
- **SaveLoadException**: Thrown by the `Serialization` module when binary save data fails magic-byte verification, size bounds checking, or XOR decryption key validation.
- **StateException**: Thrown by `StateMachine` during invalid state operations (e.g., attempting to fetch an active state from an empty state stack).

4.4. Exception hierarchy tree

```
std::exception (Standard C++ Base)
 └ GameException (Base class for all game errors)
     ├ ResourceException (Asset loading failures)
     |    ├ TextureLoadException
     |    ├ FontLoadException
     |    └ AudioLoadException
     ├ SaveLoadException (Save/Load errors)
     |    └ CorruptedSaveException (Binary/Encryption corrupted)
     └ StateException (State Machine flow errors)
          └ EmptyStateException
```

4.5. Three-tier exception handling mechanism

Our engine implements a Three-tier exception handling architecture to isolate, handle, and log errors at appropriate application layers:

- **Tier 1: Throwing layer (Low-level subsystems)**
  Low-level managers (`ResourceManager`, `Serialization`) detect hardware IO or memory anomalies and throw specific `GameException` instances with descriptive contextual messages.

- **Tier 2: Recovery & fallback layer (Mid-level engine & states)**
  Mid-level modules (`PlayingState`, `Game::loadPlayerData()`) catch non-fatal errors using local try-catch blocks. For example, if `CorruptedSaveException` is caught during save-slot loading, Tier 2 intercepts it, resets player data to default safe values, and saves a clean encrypted file without interrupting the user.

- **Tier 3: Top-level safety net (main.cpp)**
  The entry point wraps the primary `Game::run()` loop in a global Tier 3 `try-catch` block. Unhandled critical exceptions propagate to Tier 3, where a terminal error message is printed to `std::cerr` and resources are safely released before a clean application shutdown.

V. CORE PROCESSING TECHNIQUES

1. Software architecture & programming paradigms

1.1. Object-oriented programming (OOP)

Object-Oriented Programming (OOP) is a programming paradigm developed as software systems grew increasingly complex and procedural programming was no longer sufficient. Object-oriented programming models software components as real-world objects, each possessing its own distinct attributes and methods. [1]

1.1.1. Encapsulation

Encapsulation is one of the four core pillars of Object-Oriented Programming (OOP). It is a mechanism that bundles related data and behaviors together into a specific unit called a class. Concurrently, the internal data and behaviors within each class are strictly assigned appropriate access permissions.

In C++, data and behaviors are categorized into three encapsulation levels represented by the `public`, `protected`, and `private` keywords. The `public` keyword allows all other classes to access and utilize the attributes and methods of that class. The `protected` keyword permits only the class itself and its derived classes to access and utilize them. Finally, the `private` keyword limits access and usage strictly to the defining class; in other words, attributes and methods declared as `private` are completely hidden from all other classes, including its own derived classes. [1]

Adhering to encapsulation in object-oriented programming yields numerous benefits:

- First is safeguarding data integrity. Declaring class attributes within the private scope prevents external operations from modifying or assigning invalid values. [2] To access these attributes, other classes typically must utilize publicly exposed intermediary methods, such as getters and setters, which allow for validity checking prior to assignment.
- Next is flexibility when modification of the source code is required. Although the open-closed principle discourages modifying existing source code, when changes are genuinely necessary, information hiding ensures that other classes remain unaffected. This is because external classes interact exclusively with publicly exposed methods rather than directly accessing internal attributes. If everything were left `public`, any modification would necessitate recompiling and testing the entire codebase, since it would be impossible to determine the true scope of impact caused by the altered

details. [2]

- Furthermore, applying `private` to attributes enables granular access control: certain attributes can be set as read-only, write-only, read-write, or completely inaccessible. [3] This also establishes consistency throughout the codebase, as developers do not have to deliberate over when to use an interface function versus when to call an attribute directly. Consequently, attributes are conventionally declared within the `private` scope. [3]

In the Road Crossing game project, encapsulation is rigorously applied across object classes. For instance, the "`People`", "`Map`", and "`Lane`" classes communicate exclusively via `public` methods such as `die()`, `tryToMove()`, and `update()`. However, certain limitations still persist in specific areas. A prime example is within the "`Lane`" class, where the `spawnObject()` method possesses excessive knowledge about the "`Vehicle`" class and configures its detailed parameters, despite doing so through `public` interface methods.

1.1.2. Inheritance

Inheritance is a mechanism that allows the creation of a new class, known as a child class, based on an existing base class, known as a parent class. The child class typically exhibits an "is-a" relationship with the parent class and inherits members within the `protected` and `public` scopes of the parent class. Furthermore, the child class can possess its own distinct attributes and methods. [4]

The primary objectives of inheritance are code reusability and representing hierarchical relationships among classes. Instead of redundant code for shared methods across multiple classes, inheritance enables more efficient reuse. Concurrently, inheritance provides a clearer representation of how classes relate to one another within the system structure. [2], [4]

On the other hand, large-scale software systems often encounter challenges when inheriting entities form excessively deep inheritance trees with too many hierarchical layers, leading to a "class hierarchy explosion". [5] Consequently, software architects generally strive to utilize composition or aggregation ("has-a" relationships) between entities instead of "is-a" relationships during class design. This practice minimizes unnecessary over-inheritance, reinforces the principle of loose coupling among classes, and enhances encapsulation through the information hiding inherent in "has-a" relationships. [5], [6]

In this project, the inheritance relationship is most prominently manifested within the state classes and entities. The `HomeState`, `PauseState`, and `LoadState` classes inherit

from a common parent class, `State`. Similarly, game entities such as `Car`, `Truck`, and `Bird` share `Object` as their ultimate base class. This architecture effectively captures the commonalities of derived classes and enables sensible code reuse. However, a current limitation is that the `Object` class remains overly generic, burdened with numerous methods that are not universally shared by all subclasses. To resolve this, the team plans to refactor the architecture in the future by introducing two intermediate classes, `StaticObject` and `DynamicObject`. This will cleanly segregate the entity classes and prevent static elements like `Rock` or `Grass` from inheriting unnecessary methods such as `move()` and `getVelocity()`.

1.1.3. Polymorphism

Polymorphism is one of the core pillars of Object-Oriented Programming (OOP). The word "polymorphism" is a term derived from Greek, meaning "many forms". [2] This technique allows developers to abstract away the differences between sibling classes or data types of the same hierarchy, utilizing them through a unified method. Depending on the concrete object being invoked at runtime, it will respond in its own distinct manner. [2]

There are two common types of polymorphism encountered in programming: static polymorphism and dynamic polymorphism.

- Static polymorphism occurs when the mechanism for selecting the appropriate response is determined at compile time, based on the return type and parameter list to select the function to be invoked. This mechanism is achieved through function/method overloading and operator overloading. Furthermore, the template mechanism in C++ also represents a form of static polymorphism. [1], [7]
- Unlike static polymorphism, where methods or functions are resolved during compilation, dynamic polymorphism defers the binding resolution until runtime. Typically, to achieve dynamic polymorphism, a function must be declared as a virtual function and invoked via a base class pointer or reference that points to a derived class object while the program is running. [2]

The application of polymorphism makes the source code clean and elegant by allowing the reuse of generalized code structures. [1] Furthermore, polymorphism heavily supports the principle of programming to an interface, not an implementation. Additionally, leveraging polymorphism effectively enhances codebase maintainability and scalability. Instead of writing dozens or hundreds of lines of if-else or switch-case statements whenever selecting an object to execute a method, polymorphism condenses the operation into a single, simple function call. Similarly, when a new derived class needs to be added,

developers only need to implement the specific methods of that class without having to modify and append `if-else/switch-case` structures throughout the entire codebase. [6] In summary, applying polymorphism reduces systemic coupling, minimizes code redundancy, and makes the system flexible, highly maintainable, and easier to scale.

In the project, polymorphism is effectively applied to manage the game state windows. Within this architecture, the "`State`" class serves as a unified interface that exposes the `init()`, `update()`, `render()`, and `handleEvent()` methods, remaining completely agnostic to which specific state is currently active. Instead of relying on complex conditional statements or monolithic switch-case structures, the main game loop simply executes "`states.back()->update(dt);`" or "`states[i]->render(window);`". At runtime, C++ automatically resolves and dispatches the exact `update()` or `render()` method of the corresponding derived class (such as `HomeState`, `PauseState`, or `LoadState`). This dynamic dispatch grants the state management system exceptional flexibility and seamless scalability.

1.1.4. Abstraction

Abstraction is a technique that focuses on the essential characteristics of an object while ignoring complex underlying implementation details. It enables developers to model concepts without needing to concern themselves with how they are executed under the hood. [4]

This achieves a complete decoupling of interface and implementation. Instead of merely exposing or modifying data via standard getter and setter methods, a class provides abstract interfaces that allow different classes to interact seamlessly, completely agnostic to the underlying structure or complex implementations hidden beneath the surface. [2], [4]

In this project, abstraction is fundamentally embedded in the architecture through the use of abstract base classes and pure virtual functions. For example, the State class acts as a high-level interface for all game screens, declaring pure virtual methods such as `init()`, `update()`, and `render()`. This allows the central `StateMachine` to orchestrate various screens seamlessly without needing to understand the specific, complex implementations of the `PlayingState` or `SettingState`. Similarly, the `Object` class abstracts the concept of a "map entity", requiring all objects to have an `updateState()` method while leaving the concrete movement algorithms to the derived classes.

Furthermore, system-level abstraction is prominently demonstrated in the

`ResourceManager` class. Loading graphical assets from a hard drive, caching them in memory (`std::unordered_map`), and handling potential I/O errors are highly complex processes. However, ResourceManager completely abstracts this complexity away. Developers working on game entities only need to execute a simple method like `Resource().getTexture()`, completely agnostic to the underlying memory management mechanisms.

Despite these architectural strengths, the current design experiences a phenomenon known as "Leaky Abstraction", specifically within the Lane class. An ideal abstraction ensures the caller never needs to know the hidden implementation. However, because the Object interface currently lacks specific dynamic behavioral definitions (such as a generic method to retrieve current speed or movement state), the Lane class is forced to bypass the abstraction layer. It heavily relies on runtime type identification via dynamic_cast to reveal the concrete derived classes (`Vehicle` or `Animal`) to calculate obstacle avoidance logic. To resolve this tight coupling, future refactoring will introduce the `DynamicObject` interface to provide these necessary methods, thereby sealing the abstraction leak and preserving the system's integrity.

1.2. Design patterns

A Design Pattern is a proven, general solution to common problems encountered in software design.

In this course project, selectively applying Design Patterns not only fulfills technical requirements but also standardizes the codebase architecture, optimizes both system resources and performance, and resolves project-specific challenges efficiently and seamlessly.

1.2.1. Singleton

1.2.1.1. Purpose of singleton

The Singleton pattern ensures that a class has only one instance throughout the application's lifecycle while providing a global point of access to that instance.

1.2.1.2. Strengths and weaknesses of singleton

Strengths: One of Singleton's defining advantages comes from guaranteeing a single instance of a class. Having only one instance from the outset helps centralize system data, prevents logical conflicts and memory leak/overflow risks, and ultimately improves game execution speed.

Weaknesses: Due to its global accessibility, this pattern can tightly couple dependencies between objects, making unit testing more difficult.

1.2.1.3. Application in the course project

In this project, the `ResourceManager` class is designed using the Singleton pattern to serve as a Centralized Resource Repository. Without Singleton, every time a vehicle or animal spawns, it would duplicate texture data in RAM, resulting in wasted memory resources and degraded rendering performance. Because the `ResourceManager` should act as a single, once-instantiated source to keep game classes consistent, eliminate logic conflicts, and protect RAM capacity, implementing it with the Singleton pattern was the ideal choice.

To prevent multiple `ResourceManager` instances from being created, the class constructor was made `private`. Additionally, providing a single global access point ensures that the game operates coherently and remains synchronized throughout its entire lifecycle.

1.2.2. Simple Factory

1.2.2.1. Purpose of simple factory

The Simple Factory pattern encapsulates complex object instantiation logic into a dedicated class or method, effectively hiding the creation details from external client code.

1.2.2.2. Strengths and weaknesses of simple factory

Strengths: A key advantage of Simple Factory is that creating an object merely requires invoking a method within the factory class. Furthermore, it cleanly separates function invocation from the underlying algorithmic implementation, making the codebase easier for developers to modify and maintain.

Weaknesses: A major drawback is its reliance on conditional branching logic (such as `if-else` or `switch` statements), which causes the class to bloat as new object types are added. This can lead to messier code that becomes harder to manage over time.

1.2.2.3. Application in the course project

In this project, randomly spawning vehicles and animals across traffic lanes is a crucial element that dictates game difficulty and overall gameplay dynamics. To accomplish this, the project combines the standard C++ `<random>` library with the Simple Factory pattern, allowing the system to automate object creation dynamically and efficiently.

Combined with random generation, spawning new lanes as players progress through older ones becomes much simpler. This liberates the game from traditional constraints—such as fixed finish lines or linear level advancement—turning it into an endless-runner gameplay style, where players cross as many lanes as possible to achieve the highest score.

The `Lane` class acts as a Simple Factory responsible for instantiating vehicles and animals on each lane. The object creation logic is encapsulated within two methods: `spawnObject` (used to generate designated objects on the lane) and `seedObjects`. The `seedObjects` function is called whenever a new lane is generated to determine the initial count and types of entities that appear on that lane, preventing repetition and making every lane feel unique.

The `spawnObject` function serves as the core of the Simple Factory implementation. Executed during each frame update in the game loop, this method acts as a control filter for the lane. It first checks spatial constraint logic (to prevent entities from overlapping), then utilizes branching logic to determine the specific object type to create, and finally instantiates the entity onto the lane.

1.2.3. State

1.2.3.1. Purpose of state pattern

The State pattern allows an object to alter its behavior when its internal state changes, encapsulating each state into an independent class.

1.2.3.2. Strengths and weaknesses of state pattern

Strengths: The primary advantage of the State pattern is the elimination of complex conditional branching (`if-else`, `switch-case`). Modularizing state-specific behaviors makes the codebase significantly more readable, reusable, and easily extensible.

Weaknesses: It can generate a high number of classes to manage all possible states, which may increase project structure complexity as the file count grows.

1.2.3.3. Application in the course project



The title screen

In this course project, managing game states is a classic application of the State pattern in software design.

A typical game traverses multiple distinct states, such as Pause, Main Menu, Game Over, and active gameplay. Handling these states via `if-else` or `switch` statements would not only render the code unreadable, but also violate core software design principles (such as the Open/Closed Principle). The State pattern was selected specifically to resolve this issue. The project encapsulates each game state into its own dedicated class. The game loop manager acts as the Context, maintaining a pointer to the active state and delegating all event handling, state updates, and rendering directly to that state.



The pause screen

This structure allows seamless transitions; for example, during gameplay, players can switch to a paused state via the Pause button. In the Pause state, four new controls become available (Resume, Save, Settings, and Return to Main Menu). The project utilizes a State Machine to govern these transitions, while a `SharedContext` enables independent states to communicate and safely share resources through Dependency Injection.

The main benefit of applying the State pattern to game states is the clean decoupling of state responsibilities. Because states do not depend directly on one another, developers

can easily debug existing states and integrate new ones in the future (e.g., Leaderboards, Credits).

1.2.4. Polling

1.2.4.1. Purpose of polling pattern

The Polling pattern is a mechanism in which a component actively and continuously queries (checks) the state of another component or system at fixed intervals to respond promptly when changes occur.

1.2.4.2. Strengths and weaknesses of polling pattern

Strengths: This pattern is favored for its simplicity and ease of implementation. The system avoids the complexity of constructing a dedicated messaging or Event System, relying instead on straightforward conditional checks (`if-else`) placed directly inside the `update()` function. This approach is particularly effective when integrated into the Game Loop, allowing entity states to be continuously evaluated and updated on a per-frame basis.
Weaknesses: The main drawback of Polling is CPU resource wastage caused by constant checks even when states remain unchanged. When handling a large volume of objects, the exponential increase in condition checks can cause noticeable performance degradation and FPS drops.

1.2.4.3. Application in the course project

In this course project, vehicle and animal entities require real-time, continuous state updates. Rather than implementing an overly complex Event System, the team applied the Polling pattern directly within the `UpdateState(float delta_time)` method, which is invoked every frame.

The operational flow of the `UpdateState` method is structured as follows:

- Proximity Checking: In each frame, an entity actively checks whether other entities are in its immediate vicinity.
- Smooth Deceleration: Based on the current distance between entities, if another object is detected nearby, the entity smoothly decreases its velocity to prevent entities from overlapping or stacking on top of each other.
- Emergency Braking: If the predefined safety threshold distance is breached, the entity halts immediately.
- Early Exit Optimization**:** At the very beginning of `UpdateState`, the system checks

whether the entity has exited the lane. If it is out of bounds or inactive, the method returns early to skip redundant calculations and conserve processing power.

Pseudocode for updating state of a vehicle

```
ALGORITHM UpdateVehicleState(deltaTime):
    // 0. Validate conditions (Polling optimization)
    IF isOutOfLane OR NOT isActive THEN
        RETURN
    END IF

    // 1. Poll surrounding environment
    nearestTraffic = GetNearestTrafficAhead()
    nearestVehicle = GetNearestVehicleAhead()

    // 2. Calculate target speed based on safe distance
    vLight = CalculateSpeedBasedOnTraffic(nearestTraffic)
    vVehicle = CalculateSpeedBasedOnVehicle(nearestVehicle)
    vTarget = MIN(baseSpeed, vLight, vVehicle)

    // 3. Calculate smooth acceleration over time
    acceleration = (baseSpeed * baseSpeed) / safeDistance
    IF vTarget > currentSpeed AND (currentSpeed + acceleration
    * deltaTime) <= baseSpeed THEN
        vTarget = MIN(vTarget, currentSpeed + acceleration *
        deltaTime)
    END IF

    // 4. Update velocity, movement, and animation
    velocity = (vTarget * direction, 0)
```

```
24      position = position + velocity * deltaTime
25      UpdateAnimationSprite()
26  END ALGORITHM
```

1.3. Advanced memory management (smart pointers)

1.3.1. Fundamental principle of unique pointers

In C++ game development, managing computer memory efficiently is a critical requirement to ensure a smooth and stable gaming experience. To achieve this, our project implements **`std::unique_ptr`**, a modern C++ smart pointer feature - a unique pointer is designed to maintain strict, exclusive ownership of a dynamically allocated object.

This means that only one unique pointer can manage a specific piece of memory at any given time. It operates on the **RAII** (**R**esource **A**cquisition **I**s **I**nitialization) principle, ensuring that once the pointer goes out of scope or is destroyed, the memory it holds is automatically and safely released back to the system.

1.3.2. Architectural advantages in game development

The application of smart pointers is especially necessary in game development due to the dynamic nature of gameplay. In our game, the system must continuously spawn and destroy hundreds of entities—such as moving vehicles, floating logs, and coins—every minute. If using traditional raw pointers *(requiring manual* **`new`** *and* **`delete`** *commands)*, there is an extremely high risk of memory leaks, wasting the computer's RAM, causing severe frame rate drops (lag) and eventual system crashes.

By utilizing smart pointers, the game architecture automatically resolves this issue. When an obstacle moves off the screen and is removed from the game's entity list, its smart pointer is destroyed, immediately clearing the associated memory without manual intervention. Furthermore, smart pointers are highly preferred in game engines because they introduce almost zero performance overhead compared to raw pointers. They also establish a clear "ownership" structure, preventing critical bugs such as "dangling pointers", “null pointer dereference”. Ultimately, smart pointers provide both optimal performance and uncompromising memory safety.

1.4. Data structures & STL utilization

1.4.1. STL used in project

Category	STL header	Intended use
Containers	`<vector>`	Stores the stack for the State Machine, traffic, top scores, and menu UI.
	`<deque>`	Stores `Objects` in `Lane` and the list of `Lanes` in `Map`.
	`<unordered_map>`	Acts as a resource cache with `O(1)` complexity.
	`<list>`	Use for audio systems. Contains sound effects that are currently playing
Memory	`<memory>`	Uses `unique_ptr` to manage the entire lifecycle of dynamic objects, replacing raw pointers.
Algorithms	`<algorithm>`	Executes sorting and searching algorithms.
Random	`<random>`	Generates random numbers for gameplay logic (spawn rates, speeds).
I/O	`<iostream>`	Debugging/logging.
	`<fstream>`	Saves/Loads data to binary files.
Strings	`<string>`	Processes and stores character strings.
Errors	`<stdexcept>`	Throws exceptions when resource loading fails.
Utilities	`<utility>`	Provides `std::pair` and `std::move` for data manipulation.
	`<limits>`	Calculates distances (using the max float value as a sentinel).
	`<cmath>`	Supports complex mathematical calculation functions.
	`<optional>`	Handles event polling from the SFML library window.
	`<string_view>`	Helps save-game encryption system.

1.4.2. Data structure

1.4.2.1. Entity data

- The project does not use a database. Game objects (`Player`, `Vehicles`, `Animals`,

`Traffic`) are stored via Classes.

- The base `Object` class contains shared core data: coordinates, collision hitboxes, velocities, and active states.

1.4.2.2. STL data structures

- **`std::deque`** (Double-ended queue): This is the most crucial structure.
  - **Infinite scrolling**:
    In the `Map` class, `std::deque<std::unique_ptr<Lane>>` is used to spawn (`push_front`) new lanes at the top and destroy (`pop_back`) old lanes at the bottom as the camera moves, achieving `O(1)` complexity at both ends.
  - **Entity management & Object pooling**:
    `Object` management strictly operates through `push_back` (add new) and `pop_front` (remove/reuse). The update logic (detailed in ***section 3.3.3***) always ensures that the order of objects within a `Lane` is never changed. Thanks to this, the system can directly apply binary search (`std::lower_bound`) without incurring the cost of calling the `sort()` function. It simultaneously supports the *Object Pooling* algorithm (reusing objects that have moved off-screen to avoid memory reallocation).
- **`std::vector`** (Dynamic array): Used in screen (state) management to act as a Stack (`std::vector<std::unique_ptr<State>>`). It allows states to be pushed and stacked on top of each other (e.g., the pause screen overlaying the playing screen) while still supporting random access when necessary.
- **`std::unordered_map`** (Hash table): Acts as a Cache in the `ResourceManager`. Used to store resources (images, fonts, sounds), ensuring each resource is loaded into RAM only once, thus preventing memory leaks.
- **`std::list`** (Doubly linked list): Lists are sequence containers that allow constant time insert and erase operations anywhere within the sequence, and iteration in both directions. In the audio system, because these sounds can end at random times (not in order), using `std::list` allows you to remove any sound in the middle of the list as soon as it finishes playing with `O(1)` time without shifting other elements.

2. Game engine & runtime infrastructure

2.1. Game loop & delta time

2.1.1. Game loop [8]

Our game uses Game loop architecture combined with a Stack-base State Machine, a common design in 2D game development.

One of the key components of the game engine is the game loop, which controls the flow of the game in real-time. It is responsible for updating the game state, handling player input, and rendering the game to screen. The game loop is typically implemented as an infinite loop that runs continuously while the game is in progress. It repeatedly processes input, updates game state, and renders the game, providing the illusion of smooth animation and interaction.

There are several types of game loops that are commonly used in game development. Each type has its own advantages and disadvantages, and the choice of which game loop to use depends on the specific requirements of the game and the platform it is being developed for:

1. **The fixed-time step loop:** This type of game loop runs at a fixed interval, regardless of the amount of time that has passed since the last update. This ensures that the game runs at the same speed on all devices and reduces the chance of bugs caused by variable frame rates. But it can lead to issues if the game becomes too complex or if the player's device is not powerful enough to handle the fixed frame rate.
2. **The variable-time step loop:** This type of game loop updates the game state based on the amount of time that has passed since the last update. This can help to ensure that the game runs smoothly on a wide range of devices, but it can lead to issues if the frame rate drops too low.
3. **The semi-fixed time step loop:** This type of game loop combines elements of both fixed-time and variable-time step loops. It uses a fixed time step for the game logic, but adjusts the rendering based on the amount of time that has passed since the last update.
4. **The event-based game loop:** This type of game loop does not run continuously; instead, it waits for specific events to occur (i.e., user input or an update from the server) and then processes them. This approach can be useful for games that require a lower update rate.
5. **The Asynchronous game loop:** This type of game loop allows the game to update and render in an asynchronous fashion, meaning that the game can continue to update

and render even while waiting for input or other resources to load. This approach can help to reduce the perceived latency and make the game feel more responsive.

A game loop (or main loop) is typically an infinite loop. A traditional game loop can be broken up into the following parts:

1. **Input handling:** The game loop starts by handling any input from the player, such as keyboard, mouse, or gamepad input. This step is responsible for updating the game state based on the player's actions.
2. **Updating game state:** The game state is updated based on the input received in the previous step, as well as any other factors that affect the game, such as physics and AI. This step is responsible for updating the position of characters, checking for collisions, and updating the game's internal logic.
3. **Rendering graphics:** The game loop then renders the updated game state to the screen. This step is responsible for drawing the characters, objects, and environments to the screen.
4. **Wait/Sleep:** After the graphics have been rendered, the game loop waits for a specific amount of time, known as the frame rate, before starting the next iteration. This helps to ensure that the game runs at a consistent speed on different devices

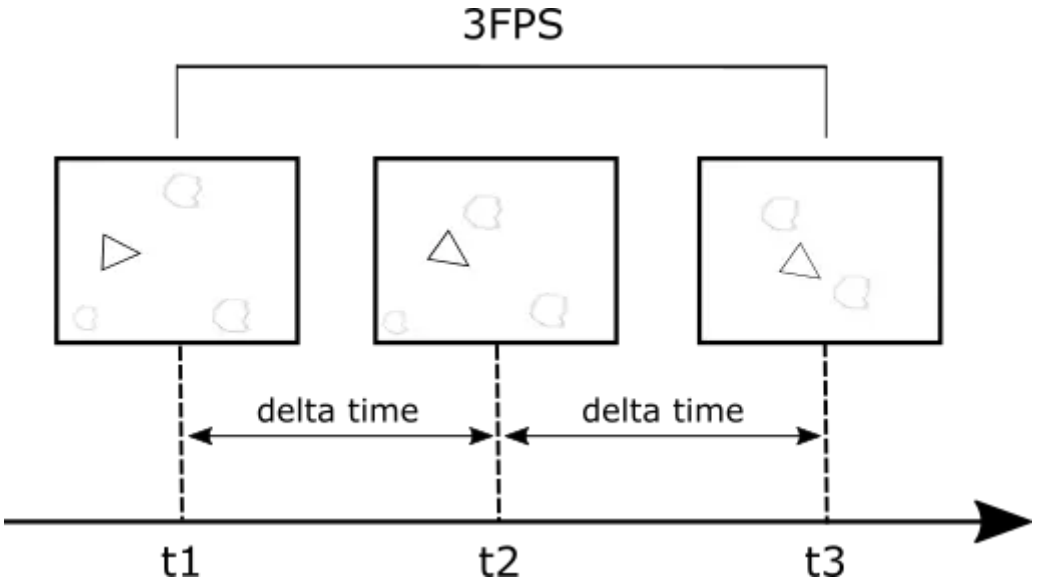
The game loop then repeats these steps, over and over again, until the game is closed. It is worth noting that the order of the steps can vary depending on the game and the game engine used. For example, some game engines may handle physics and collision detection on a separate thread. This is known as multi-threading.

Our game loop is implemented in the following order: process input → update game state → render graphics → wait, and is expected to utilize multi-threaded processing: collisions between objects, changes in object speed, etc.

2.1.2. Delta time [9]

The game runs at different device speeds depending on the CPU speed. This is normal for game genres that don't rely on FPS. However, this game is a variable-time step with a framerate cap, developed to depend on FPS because the movement speed of objects needs to be the same on different devices. Delta time is the solution to solving our problem, freeing our code from the frame rate, making it frame rate independent.

Delta time, or also referred to as elapsed time, is usually a value that is calculated for us within modern game engines such as Unity or Unreal. Delta time describes the time difference between the previous frame that was drawn and the current frame.



This value should be used when dealing with parts of our code that need to run independently of the frame rate. Where something needs to be done over some specified amount of time, such as travelling a certain distance in a second.

2.2. Window management and adaptive resolution handling

2.2.1. Architectural advantages

In 2D arcade game development, decoupling the logical game resolution from the physical display window is a critical architectural requirement. Rather than locking the application to a rigid, fixed pixel size or allowing arbitrary window stretching, our system combines a Virtual Viewport (`sf::View`) with Adaptive window scaling. This approach provides three primary technical benefits:

- **Aspect ratio preservation & visual integrity:** Locking the logical resolution to 1400 x 1050 *(4 : 3 aspect ratio)* guarantees that all graphical assets (sprites, tilemaps, HUD) render without distortion or pixel stretching across different monitors.
- **Gameplay fairness & Anti-cheat:** For grid-based arcade mechanics, the player's visible field-of-view directly determines game difficulty. Fixing the logical camera view prevents players from resizing the window to gain an unfair sightline advantage over upcoming obstacles.
- **OS interoperability & screen bounds safety:** On smaller screens (e.g., laptops with 1080p displays), a static 1050px height window can spill past screen boundaries or get hidden behind the OS Taskbar. Dynamically capping the physical window size to $\leq$ 85% of the desktop height ensures full visibility while preserving the 4 : 3 aspect ratio.

2.2.2. Core implementation

```
1   const sf::VideoMode desktop = sf::VideoMode::getDesktopMode();
2   unsigned int win_width = Constants::WINDOW_WIDTH;
3   unsigned int win_height = Constants::WINDOW_HEIGHT;
4
5   const float max_ratio = 0.85f;
6   const unsigned int max_safe_height = static_cast<unsigned int>
7   (desktop.size.y * max_ratio);
8
9   if (win_height > max_safe_height) {
10      win_height = max_safe_height;
11      win_width = static_cast<unsigned int>(win_height
12   * (4.0f / 3.0f));
```

```
13  }
14
15  window.create(sf::VideoMode({win_width, win_height}),
16  Constants::GAME_TITLE, sf::Style::Titlebar | sf::Style::Close);
17
18  sf::View default_view(sf::FloatRect({0.f, 0.f},
19  {1400.f, 1050.f}));
20  window.setView(default_view);
```

2.2.3. Technical breakdown

2.2.3.1. Desktop bounds detection & safe-scaling logic

The application queries the user's display environment using sf::VideoMode::getDesktopMode(). If the default 1050px height exceeds 85% of the detected screen height (desktop.size.y * 0.85), the physical window dimensions are scaled down proportionally. The horizontal width is recalculated using the constant 4:3 multiplier (`W = H × 4 / 3`), ensuring the titlebar and window frame fit entirely within the desktop work area.

2.2.3.2. Virtual viewport mapping (`sf::View`)

By passing a fixed 1400×1050 `sf::FloatRect` into `sf::View`, SFML automatically handles hardware-accelerated scaling to fit the physical window size. All game physics, entity coordinates, map rendering, and HUD overlays operate exclusively within this static 1400×1050 logical coordinate space, eliminating the need to recalculate object positions when running on lower-resolution screens.

2.2.3.3. Input coordinate synchronization (`mapPixelToCoords`)

Because the physical window size can differ from the 1400×1050 logical canvas, raw mouse pixel events (`sf::Event::MouseMoved, sf::Event::MouseButtonPressed`) do not match logical UI coordinates. To resolve this, all UI event handlers utilize `window.mapPixelToCoords(pixel_pos)`, which maps screen pixel clicks back to the exact 1400×1050 logical coordinate space. This guarantees 100% accurate UI button hovering and clicking across all screen scale factors.

3. Real-time game processing & mechanics

3.1. Input processing

3.1.1. The principle behind input processing

When the game is launched, a single game loop runs on the main thread; each frame it sequentially polls input, updates game logic with the elapsed delta time, and renders graphics.

Since this project utilizes SFML, capturing keyboard data through conventional functions like `getch()` is highly discouraged; without active player input, such functions would cause the main thread to block and remain idle, halting the entire loop while waiting for a keypress. Therefore, choosing SFML for this project is highly appropriate, as it is a library that excels not only in professional audio and graphics rendering but also in processing player input with high efficiency.

To process player input, the main thread first needs to open a window using the SFML library. Following this, the main thread calls `window.pollEvent()` to obtain an `sf::Event`; this function is non-blocking, which would otherwise force the main thread to wait and cause the game screen to freeze. Instead, it utilizes an Event Polling model to continuously check for and retrieve real-time player interactions and keypresses. Consequently, while the game window is active, an inner `while` loop is executed to check exactly what input the player has provided. If the player provides no input, the game continues to update seamlessly, ensuring that players can still see the vehicles and animals moving on screen.

Once the player presses a key, the inner `while` loop processes the event and evaluates the conditions (for example, verifying if the input matches movement keys defined by SFML constants, such as `sf::Keyboard::Key::A`). Once the input conditions are met, the game state is updated accordingly outside of this event loop.

Even if a player holds down a key continuously, the inner loop concludes rapidly so that the main game can proceed with its updates. This efficiency is due to the computer processing player input at extreme speeds, adhering to the principle of batch event processing.

3.1.2. Application to the project

In this course project, players are required to use both keyboard inputs and mouse clicks on screen buttons; thus, the game must handle both keyboard and mouse events concurrently.

The SFML library effectively resolves this requirement through its built-in data types, utilizing `sf::Event::KeyPressed` to capture key presses, alongside `sf::Event::MouseButtonPressed` to detect mouse click states.

Pseudocode:

```
std::optional<sf::Event> event;
while(event = window.pollEvent())
{
    if(event->is<sf::Event::Closed>())
        window.close();

    if(const auto* key = event->getIf<sf::Event::KeyPressed>())
    {
        if(key->code == sf::Keyboard::Key::W)
            {/*Execute the function key*/}
    }

    if(const auto* mb = event->getIf<sf::Event::MouseButtonPressed>())
    {
        if(mb->button == sf::Mouse::Button::Left)
        {
            sf::Vector2i mousePos = sf::Mouse::getPosition(window);
            /*Check if the mouse is within any button*/
        }
    }
}
```

3.2. Collision processing

3.2.1. Introduction

For the course project, implementing collision detection between the player and vehicles or animals is crucial, as it determines the player's survival status while the game is running. Since the project is a 2D pixel-art game, the Axis-Aligned Bounding Box (AABB) algorithm is a classic and effective approach utilized to handle collision detection between the player and entities.

While Axis-Aligned Bounding Box (AABB) collision detection algorithms can vary based on their chosen reference points-such as the central point of an object's bounding rectangle-this project adapts to the native coordinate mechanics of the SFML library. Instead of the classical center-based approach, SFML utilizes the top-left corner as the primary reference point to evaluate bounding box intersections.

3.2.2. The principle behind the AABB algorithm

Note on Coordinate System:

It is crucial to note that in computer graphics, the screen coordinate system defines the x-axis as increasing from left to right, whereas the y-axis increases from top to bottom.

Mathematical Notations:

The width and height of the bounding rectangles are projected onto the Ox and Oy axes, respectively (where Ox increments from left to right, and Oy increments from top to bottom).

w_1, h_1: The width and height of the bounding rectangle for Object 1.

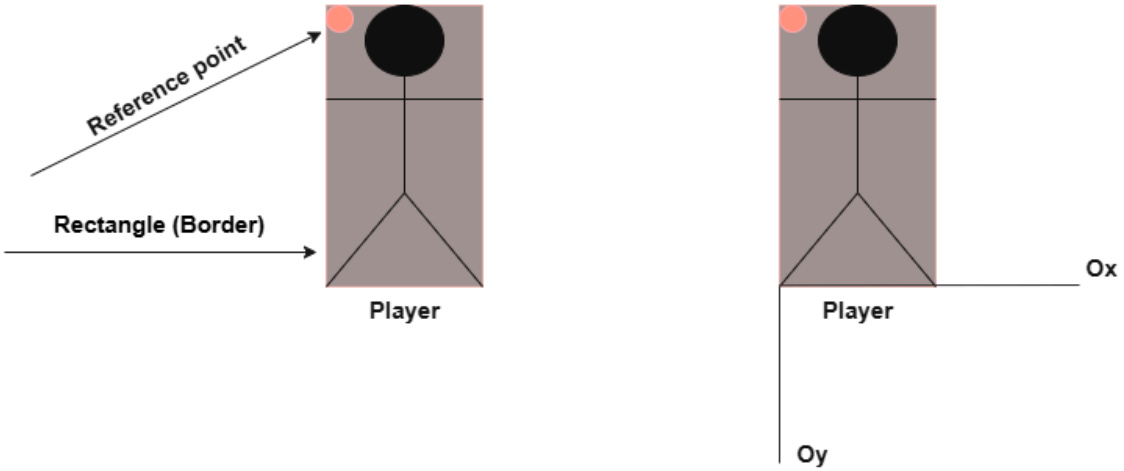
x_1, y_1: The x and y coordinates of the reference point (top-left corner) for Object 1.

w_2, h_2: The width and height of the bounding rectangle for Object 2.

x_2, y_2: The x and y coordinates of the reference point (top-left corner) for Object 2.

W,H: The width and height of the overlapping rectangle formed by the collision between the two object boundaries.

Working Principle:

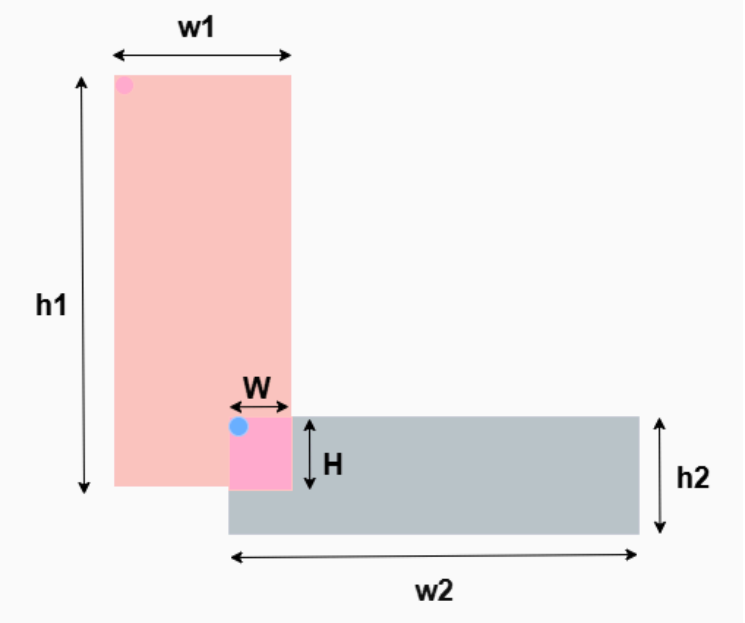


The fundamental principle of this collision detection algorithm relies on selecting the top-left corner of each pixel-based object (e.g., player, vehicle) as its reference point. This reference point is defined as the vertex located at the leftmost and topmost position of the object's bounding rectangle. Establishing these reference points is essential for applying the mathematical formulation that determines whether a collision has occurred.

Collision Condition:

A collision between two bounding rectangles occurs if and only if they overlap within the Ox and Oy coordinate space. In other words, the algorithm determines that a collision has taken place when both the width (`W`) and the height (`H`) of the intersecting rectangular region are strictly greater than zero (i.e., holding positive values).

Condition: `W > 0 && H > 0`



To fully comprehend the nature of this condition, we begin by analyzing the reference point of each entity. We define the top-left boundary of the potential overlapping region as: $X_{left} = max(x_1, x_2)$ and $Y_{top} = max(y_1, y_2)$. If a collision occurs, the coordinates (X_{left}, Y_{top}) will represent the top-left corner of the intersecting rectangle. However, this alone does not definitively prove the collision condition. Therefore, we further define the bottom-right boundary of the region as:

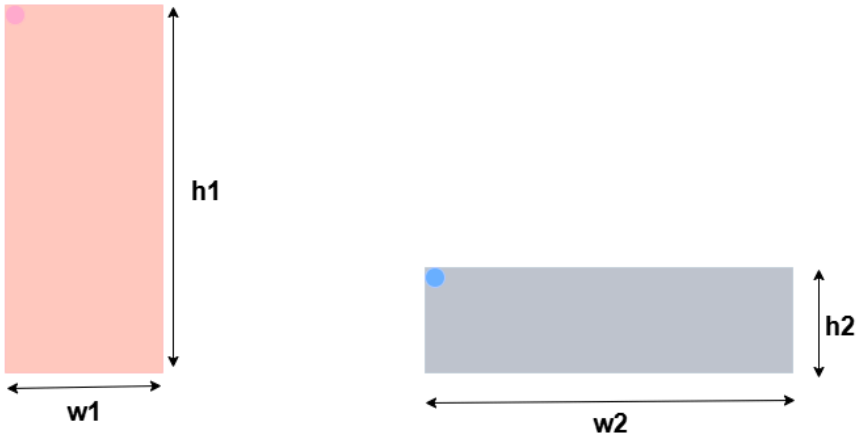
$$X_{right} = min(x_1+w_1, x_2+w_2) \text{ and } Y_{bottom} = min(y_1+h_1, y_2+h_2)$$

Consequently, the dimensions of the overlapping rectangle, namely the width (W) and the height (H), can be calculated as follows: $W = X_{right} - X_{left}$ and $H = Y_{bottom} - Y_{top}$. As a corollary, a valid collision exists if and only if this intersecting rectangular area possesses valid dimensions, satisfying the condition: `W > 0 && H > 0`

Non-Collision Case:

A non-collision state between the bounding rectangles of two entities occurs if and only if they do not overlap along either the Ox or Oy coordinate axis. In other words, the algorithm determines that no collision has taken place when either the width (W) or the height (H) of the intersecting rectangular region is less than or equal to zero.

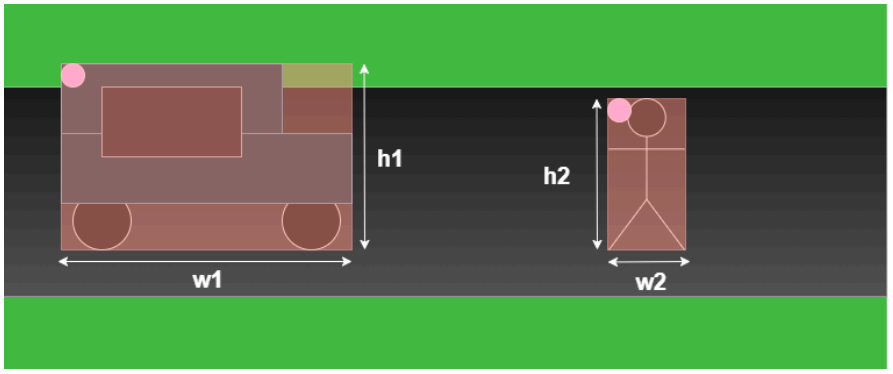
Condition: `W ≤ 0 || H ≤ 0`



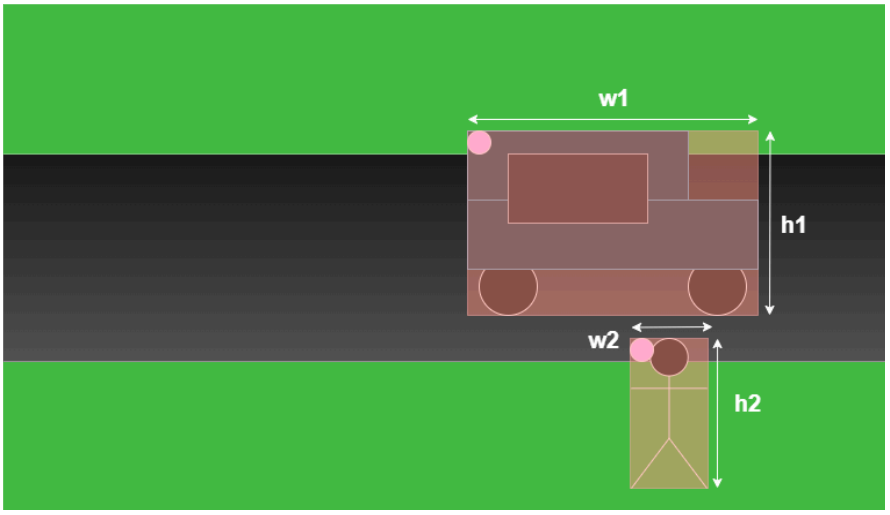
3.2.3. Application to the project

The course project will utilize the SFML (Simple and Fast Multimedia Library) to implement the collision detection algorithm between the player and other game entities (such as vehicles, animals, and obstacles).

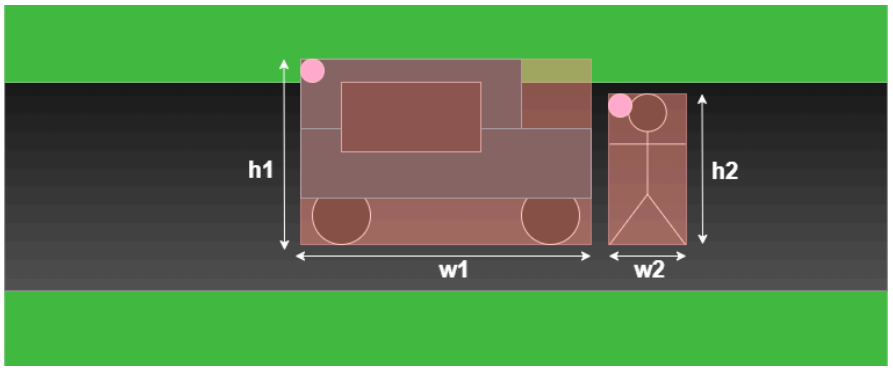
To execute this detection, the algorithm requires identifying the top-left reference point of each object, alongside the width and height of its bounding box. Consequently, the intersection between the bounding rectangles of the two objects is evaluated using the built-in collision check functions provided directly by the SFML library.



Scenario 1: The illustration demonstrates a scenario where the character and a vehicle occupy the same traffic lane. Although the collision condition along the Oy-axis is satisfied, the condition along the Ox-axis is violated; consequently, the algorithm does not report a collision.



Scenario 2: If the player is in a different lane but the vehicle is approaching the player along the Ox-axis, the algorithm will still yield a negative result (no collision). In this case, while the collision condition along the Ox-axis is met, the condition along the Oy-axis remains unsatisfied.



Scenario 3: As observed from the diagram, the vehicle is approaching the player while they are sharing the same lane. The distance between the two entities along the Ox-axis will progressively decrease until the condition is fully met, at which point the algorithm will signal that a collision has occurred.

3.2.4. Rock collision

3.2.4.1. Overview and design principle

In the game, **Rocks** act as impassable environment obstacles. Unlike lethal entities (such as vehicles or wild animals) that cause the game to end upon collision, rocks exert a blocking effect (`Constants::ObjectEffect::Blocked`).

To handle this effect cleanly without causing visual glitches or trapping the player inside geometry, the system utilizes **Predictive Collision Detection** (also known as Speculative Collision Testing). Rather than moving the player first and resolving overlaps afterward, the system predicts the player's candidate position and tests it against the environment before committing to the movement.

3.2.4.2. Predictive collision mechanism

The player's movement system combines **discrete grid-based logic** with **continuous visual interpolation (Lerp)**. To guarantee that every hop lands cleanly on a grid cell without mid-air collisions or input glitches, the game executes a 4-step predictive validation flow during every movement attempt:

- **Step 1: Animation gate (Input lock)**

When the player presses a movement key (W, A, S, D or Arrow keys), the system checks the `is_animating` flag. If the character is currently mid-hop (interpolating between two grid cells), new inputs are temporarily suppressed. This guarantees that every jump is atomic and prevents the player from queuing illegal diagonal or multi-cell movements.

- **Step 2: Logical target prediction**

Instead of relying on `position` (which represents the character's changing visual coordinate during a hop), collision predictions are computed relative to `target_position` (the character's logical grid destination). The system projects the candidate coordinates (dx,dy) forward to construct a candidate bounding rectangle (`future_hitbox_rect`).

- **Step 3: Screen boundary and static obstacle validation**

The projected `future_hitbox_rect` undergoes two sequential validation checks:

1. **Screen viewport bounds:** Verifies that the candidate position remains strictly within the playable viewport width (`GAME_PLAY_BAR_WIDTH` to `WINDOW_WIDTH`) and camera height limits.
2. **Static obstacle query:** The candidate rectangle is passed to `Map::checkBlockedCollision`. If the target cell intersects a static obstacle marked as blocked (such as a `Rock` or `Bush`), the query returns `true`, immediately rejecting the movement.

- **Step 4: State commitment and visual trajectory setup**

If the target cell is verified to be empty:

  - **Logical progress**: `target_position` is updated to the new grid coordinate.
  - **Trajectory capture:** The current visual position is stored as `start_lerp_position`, and the parabolic jump arc parameters (`jump_start_pos` and `jump_end_pos`) are initialized.
  - **Feedback & animation trigger:** The jump sound effect (`SFX::Jump`) is played, `is_animating` is set to `true`, and the character enters the preparation animation state (`AnimState::Prep`). Over subsequent frames, `updateState()` smoothly interpolates the character's visual sprite toward `target_position`.

3.2.5. Coin collision

3.2.5.1. Overview and interaction model

Unlike rocks (which use predictive collision to block movement) or vehicles (which use lethal collision to trigger game over), **Coins** utilize a **Post-Occupancy Overlap Detection** model. Coins are non-blocking, non-lethal static entities that reside in the game world until collected.

When the player character lands on a grid cell containing an uncollected coin, the collision system registers the overlap, triggers a visual and auditory feedback response, and updates the persistent currency system.

3.2.5.2. Coin collection game flow

The collection sequence is processed during each frame's update cycle (`PlayingState::update`):

- **Step 1: Spatial occupancy query**

After the player moves or updates position, `PlayingState` passes the player's bounding box (`player_->getHitBox()`) to `Map::checkCoinCollision()`. The `Map` forwards this request to all active `Lane` instances on screen.

- **Step 2: Lane infiltration & type filtering**

Each `Lane` iterates through its list of static objects (`list_static`). Using `dynamic_cast`, the lane identifies `Coin` instances and tests two conditions:

1. The coin has not already been collected (`!coin->isCollected()`).
2. The player's bounding box intersects the coin's hitbox (`coin->isColliding(player_hitbox)`)

```
int Lane::checkCoinCollision(const sf::FloatRect &player_hitbox)
{
    int collected_count = 0;
    for (auto &s : list_static)
    {
        if (auto coin = dynamic_cast<Coin*>(s.get()))
        {
           if (!coin->isCollected() &&
                                    coin->isColliding(player_hitbox))
               {
                   coin->collect(); // Mark as collected
                   collected_count++;
               }
```

```
13          }
14      }
15       return collected_count;
16  }
```

- **Step 3: State transition and visual suppression**

  Calling `coin->collect()` sets the coin's internal boolean flag `is_collected_ = true`. During the rendering pass (`render()`), coins with (`is_collected_ == true`) are bypassed by the drawing pipeline, instantly removing them from the display.

```
1   void Coin::render(sf::RenderWindow &window) const
2   {
3   if (!is_collected_)
4       {
5           StaticObject::render(window);
            // Render only if uncollected
6       }
7   }
```

- **Step 4: Currency progression, HUD refresh, and persistence**

  The total number of newly collected coins in the frame is returned to `PlayingState`:

  - **Global counter:** The global currency variable `context_->data->total_coins` is incremented.
  - **HUD refresh:** The text display (`coin_text_`) is updated dynamically to reflect the new balance.
  - **Auditory feedback:** A pickup sound effect (`SFX::Coin`) is triggered.
  - **Encryption & Persistence:** When the player saves or exits the application, `Game::savePlayerData()` encrypts `total_coins` using **XOR cipher** and writes it to `data/player_data.bin`, ensuring permanent cross-session currency retention.

3.3. Movement & traffic logic (steering behaviors)

In the project, `Vehicles` and `Traffic` on the same `Lane` always move in the same direction. Each Vehicle has a default maximum speed of `base_speed` and a safe distance of `safe_distance = base_speed * 2`.

The acceleration for a `Vehicle` to speed up from a stationary state `0` to `base_speed` within the `safe_distance` is determined by the formula:

$$\texttt{acceleration} = (\texttt{base_speed}^2 - 0^2) / (2 * \texttt{safe_distance})$$

The state of the `Vehicle` is continuously updated after each `delta_time` interval. The current speed (`cur_speed`) and previous speed (`prev_speed`) are strictly limited within the range from `0` to `base_speed`.

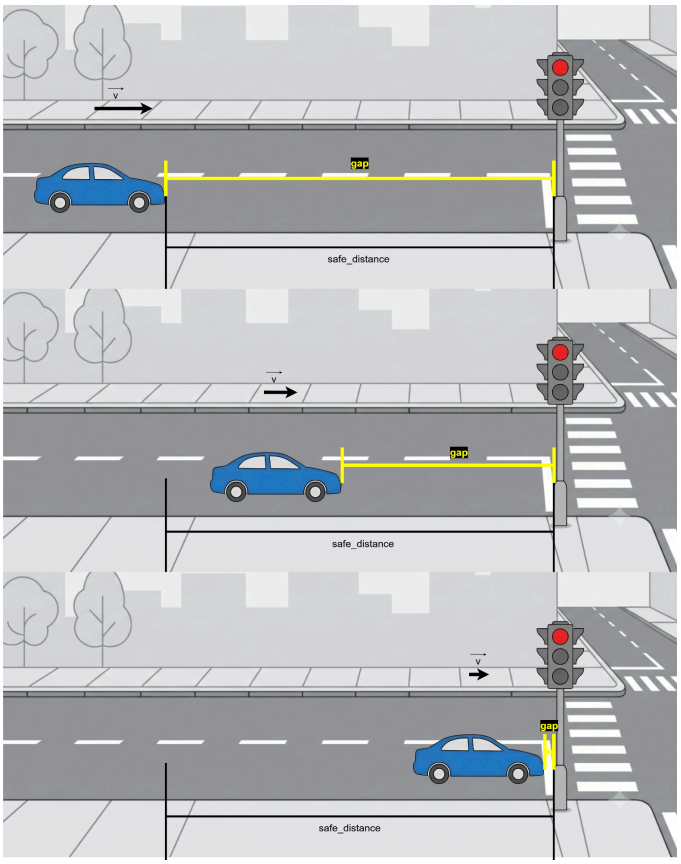
3.3.1. Deceleration logic when encountering traffic lights

At each `updateState` cycle, the `Vehicle` will scan for the nearest `Traffic` ahead within the `safe_distance`.

- If the signal light is Green or Yellow but the remaining time is sufficient to pass, the `Vehicle` will ignore this `Traffic` and continue to search for the next nearest `Traffic` within the `safe_distance`.
- If the signal light is Red or Yellow and the remaining time is insufficient to pass, the `Vehicle` is forced to decelerate and come to a complete stop right in front of the `Traffic`.

Instead of using the traditional physics formula dependent on time ($v = v_0 + a * t$) which requires additional costs for storing time variables and complex calculations, we interpolate the speed directly based on the actual distance (`gap`) from the `Vehicle` to the `Traffic`. As the `Vehicle` gets closer to the `Traffic` (`gap` approaches `0`), the speed decreases proportionally:

$$\texttt{cur_speed} / \texttt{base_speed} = \texttt{gap} / \texttt{safe_distance}$$
$$\Rightarrow \texttt{cur_speed} = (\texttt{base_speed} * \texttt{gap}) / \texttt{safe_distance}$$



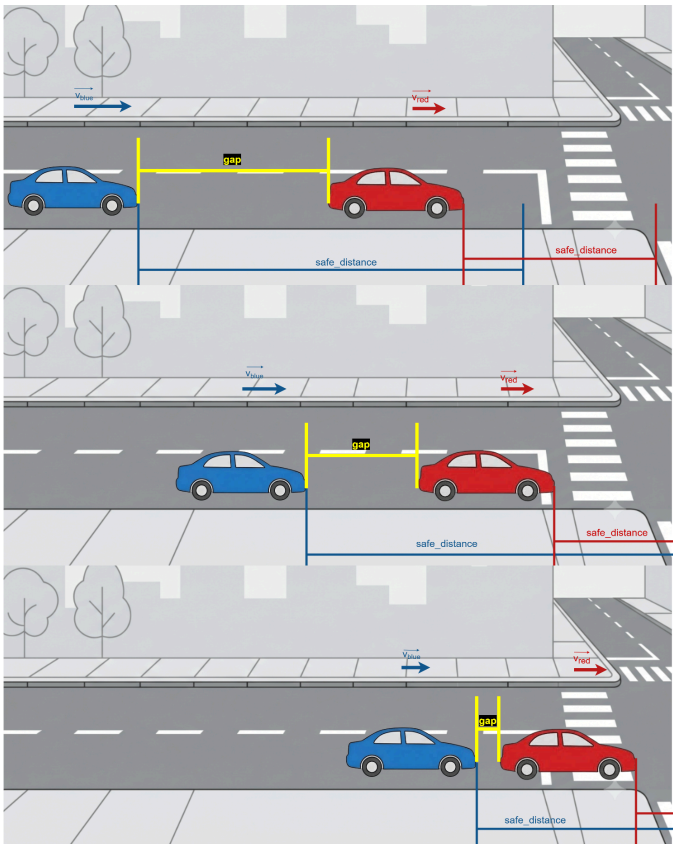
3.3.2. Logic for maintaining a safe distance from the vehicle ahead

Due to space limitations, `Vehicles` are not allowed to overtake each other. The `Vehicle` behind (suppose it is `blue_car`) will prioritize checking the nearest `Vehicle` ahead (`red_car`) within the `safe_distance` limit.

- If `cur_red_speed >= prev_blue_speed`, the `blue_car` continues to move normally because the distance between the 2 `Vehicles` is being maintained or widened.
- If `cur_red_speed < prev_blue_speed`, the blue_car will automatically brake. The system will calculate so that the speed of the blue_car gradually decreases and balances out to exactly match the speed of the red_car when closely approaching the rear of the car ahead.

The formula to determine the speed of the `blue_car` at this moment is:

```
cur_blue_speed =
        cur_red_speed + (blue_base_speed - cur_red_speed) * (gap/blue_safe_distance)
```



3.3.3. Acceleration logic upon departure

When stopped at a red light, `prev_speed = 0`. As soon as a green light signal is received, the system bypasses the deceleration logic and begins applying acceleration to smoothly speed up the car, avoiding jerky movements:

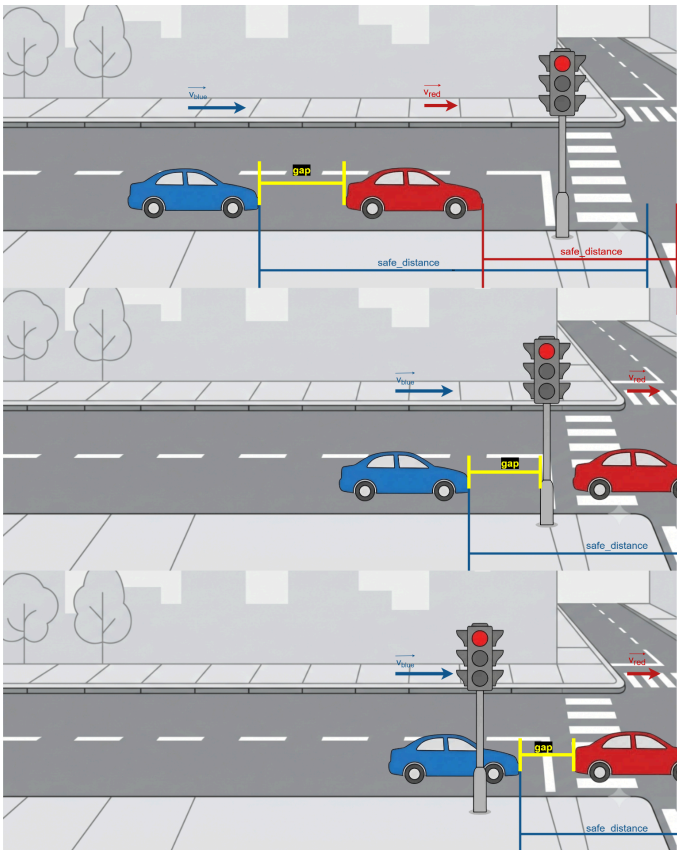
```
cur_speed = prev_speed + acceleration * delta_time
```

(maximum limit not exceeding `base_speed`).

Thus, the speed of the `Vehicle` will increase steadily from `0` to `base_speed` (or close to `base_speed` due to error margins) without requiring overly complex calculations.

3.3.4. Comprehensive algorithm (combining 3 logics)

In reality, a `Vehicle` can be simultaneously affected by `Traffic`, the car ahead, and the acceleration state.



To resolve this, we independently calculate the ideal speed when encountering `Traffic` (`v_traffic`) and when encountering the car ahead (`v_vehicle`). Because it is not constrained by `prev_speed` at the initial evaluation step, the algorithm will select the obstacle causing the greatest hindrance (the lowest speed value) to prioritize processing.

If there is no `Traffic` affecting the current speed, then `v_traffic = base_speed`, similarly for `v_vehicle`.

Pseudocode:

```
float v_traffic = base_speed
float v_vehicle = base_speed

// Check the influence of traffic lights
find nearest & available Traffic()
if(have Traffic):
```

```
6       float gap = distance between Vehicle and Traffic
7       v_traffic = base_speed * (gap / safe_distance)

8   // Check the influence of the vehicle ahead
9   find nearest & available otherVehicle()
10  if(have otherVehicle):
11      float gap = distance between Vehicle and otherVehicle
12      v_vehicle = cur_other_speed + (base_speed - cur_other_speed)
        * (gap / safe_distance)

13  // Merge speed barriers
14  cur_speed = min(base_speed, v_light, v_vehicle)

15  // Control acceleration (Avoid sudden acceleration due to unexpected
    gap changes)
16  if (cur_speed  > prev_speed
    && (prev_speed + acceleration * delta_time) <= base_speed):
17      cur_speed = min(cur_speed, prev_speed + acceleration *
        delta_time)
18  set new speed (cur_speed)
19  Vehicle.move()
```

The combination of the `min()` filter and the acceleration condition ensures the system automatically finds the safest velocity: the Vehicle never accelerates irrationally, never runs a red light, and never crashes into the car ahead, creating a natural traffic flow effect.

3.4. Camera tracking & auto-scroll system

The in-game camera is implemented using a `sf::View` whose vertical center coordinate is updated after every `delta-time`. The camera is not rigidly locked to `Player` (i.e., directly setting the view's center to the position of the `Player`) but is instead treated as an object with its own velocity and acceleration, chasing a target position (which may or may not be `Player`).

The Camera's movement logic is similar to the logic of the `Vehicle` presented in the previous section (V.3.3 Movement & traffic logic). A `Vehicle` approaching a red light does not stop immediately, and when the light turns green, it does not accelerate immediately either; instead, it derives its current velocity from the remaining distance to the target.

- `base_scroll_speed` is the floor speed, the maximum speed the Camera is allowed to reach when following the `Player`. Defined as a fixed constant.
- `scroll_velocity` is the Camera's velocity in the previous frame.
- `distance` is the remaining distance the Camera must travel to catch up with the `Player`, calculated using the formula:

```
distance = center - target_center
```

where `target_center` is exactly the position of the `Player` in the current frame.

Note that in SFML, the `y` axis increases downward. As the `Player` moves forward, `y` decreases, and as the Camera scrolls up, `center` decreases.

3.4.1. Player-tracking camera logic

In the default mode, the Camera remains stationary until the `Player` passes the center of the view (`center`). If `target_center > center`, the `Player` is still in the lower half of the screen, so the Camera sets `target_center = center` to ensure `distance` stays positive.

Once the Player passes the center, `distance > 0` and the Camera starts chasing it. The challenge is choosing a velocity that moves just enough to follow the `Player` without causing frame jitter.

At this point, the acceleration is chosen so that the Camera speeds up from a standstill to `base_scroll_speed` over a specific distance (currently set in-game to 3 times

`LANE_HEIGHT`), just enough for the Player to perceive the Camera as smoothly accelerating rather than jerking suddenly:

$$\texttt{acceleration} = \texttt{base_scroll_speed}^2 \;/\; 3 * \texttt{LANE_HEIGHT}$$

With that acceleration, we use a physics formula to compute the velocity for the current frame. An object moving at velocity `target_speed` must come to a stop exactly after traveling a distance of `distance` under a constant deceleration:

$$\texttt{target_speed}^2 = 2 * \texttt{acceleration} * \texttt{distance}$$
$$\Rightarrow \texttt{target_speed} = \texttt{sqrt}(2 * \texttt{acceleration} * \texttt{distance})$$

This formula ensures the Camera stops exactly at the target instead of overshooting `target_center` and having to move back. As `distance` grows larger, `target_speed` grows larger too and the Camera chases faster; as `distance` decreases toward 0, `target_speed` also automatically approaches 0.

However, `target_speed` still cannot be assigned directly to the Camera. Because `target_center` of the Camera is not fixed and changes with the position of the `Player` every frame, in each different frame, `distance` can change differently, causing `target_speed` to change along with it. Therefore, the velocity for the current frame needs to be further computed from the previous frame's velocity to ensure the velocity doesn't change too much between 2 frames (causing frame jitter). We specify that, each frame, the velocity is only allowed to change by at most an amount of `acceleration * delta_time`:

```
scroll_velocity += clamp(target_speed - scroll_velocity,
                  -acceleration * delta_time, acceleration * delta_time)
```

Once the velocity is obtained, the view's center is shifted by a corresponding amount:

```
center = center - scroll_velocity * delta_time
```

The result is a continuous accelerate-hold-decelerate motion, stable no matter how fast the `Player` presses keys.

Pseudocode:

```
center = view.center.y
target_center = Player.position.y
if (target_center > center):
    target_center = center

distance = center - target_center
acceleration = base_scroll_speed² / 3 * LANE_HEIGHT
target_speed = sqrt(2 * acceleration * distance)
scroll_velocity += clamp(target_speed - scroll_velocity,
                -acceleration * delta_time, acceleration * delta_time)
center -= scroll_velocity * delta_time
view.center.y = center
```

3.4.2. Auto-scroll logic combined with Player tracking

When auto-scroll mode is enabled, the Camera continuously scrolls up on its own to ensure the `Player` does not stand still for too long.

At this point, we introduce the quantity `floor_speed`, i.e. the speed floor that the Camera is never allowed to fall below. This floor increases with the `level` of the stage:

```
floor_speed = AUTO_SCROLL_BASE_SPEED * (1 + level * AUTO_SCROLL_LEVEL_FACTOR)
```

where `AUTO_SCROLL_BASE_SPEED` and `AUTO_SCROLL_LEVEL_FACTOR` are 2 fixed constants.

The tracking logic described in the previous section still runs in this mode to give the `Player` the best experience (so it doesn't have to wait for the Camera to move if the `Player` can move faster). The Camera still computes `distance`, still derives `target_speed` from the remaining distance, and still limits how much the velocity can change per frame. The difference is that the final result is clamped from below by the speed floor:

```
scroll_velocity = max(scroll_velocity, floor_speed)
```



Auto-scroll mode

As a result, when the `Player` moves slowly or stands still, the tracking part produces no significant velocity, and the Camera scrolls at exactly the `floor_speed` of the `level`. When the `Player` moves fast, `distance` grows larger, `target_speed` rises above the floor and the Camera speeds up to keep pace, revealing more view ahead so the `Player` doesn't die from lack of visibility.

Pseudocode:

```
center = view.center.y
target_center = Player.position.y
if (target_center > center):
    target_center = center

distance = center - target_center
```

```
6   acceleration = base_scroll_speed² / 3 * LANE_HEIGHT
7   target_speed = sqrt(2 * acceleration * distance)
8   scroll_velocity += clamp(target_speed - scroll_velocity,
                -acceleration * delta_time, acceleration * delta_time)
9   floor_speed = AUTO_SCROLL_BASE_SPEED * (1 + level *
                                            AUTO_SCROLL_LEVEL_FACTOR)
10  scroll_velocity = max(scroll_velocity, floor_speed)
11  center -= scroll_velocity * delta_time
12  view.center.y = center
```

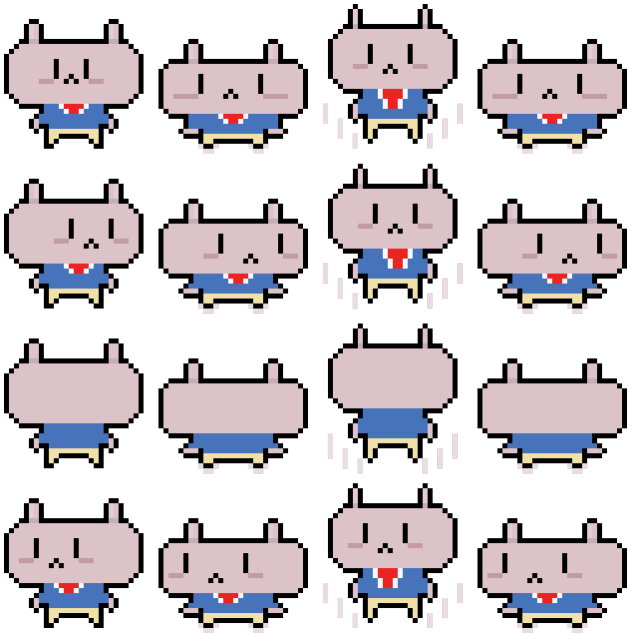
3.5. Entity animation & sprite sheet system

In this course project, building and designing animations for characters and entities is an essential task aimed at enhancing the user experience. Without animations, the game would become static (for example, the player moving in a statue-like posture), dull, and devoid of life. Furthermore, gameplay would become more difficult because players wouldn't be able to tell whether an entity is actually moving over time (for instance, without vehicle exhaust smoke effects, it is hard to tell if a car is in motion).

Designing and applying animations brings the project to life and makes it more enjoyable, as character and entity movements provide immediate feedback to players, creating a sense of responsiveness and satisfaction. Additionally, thanks to animations, the characters and entities within the project can express distinct personalities and traits, rather than remaining plain, lifeless pixel blocks.

3.5.1. Sprite sheet and frames

Using multiple individual PNG files to animate characters and entities can complicate resource management when dealing with a large number of images, while also degrading hardware performance due to frequent texture switching (leading to FPS drops). Individual PNG files should generally only be used for background elements or static entities. Therefore, an optimal solution to this issue is adopting sprite sheets, which consolidate multiple animation frames into a single image, allowing the animation behavior to be defined and rendered programmatically.



Sprite sheet for one of the characters

Regarding graphics, the team standardized the drawing and export of frames in the Piskel tool at a detailed resolution of `800 x 800` pixels. Keeping this original resolution helps ensure the clarity of the source images when working in the design software. When loaded into the game, the object is scaled down to its own in-game size and aligned to the `105 x 105` tile grid (`CELL_SIZE`).

Although sprite sheets come with their own drawbacks—such as maximum texture size limits (which occur when packing too many frames) and the inconvenience of having to re-export the entire sheet just to tweak minor details—they remain the standard for larger projects due to clean and efficient asset management. Since this project features numerous characters and entities, each requiring custom animations with multiple frames, utilizing sprite sheets is the optimal approach.

3.5.2. Program entity animation with C++

Once the sprite sheet is configured for each object, the next crucial step is integrating it into C++. After importing the sprite sheets into the project, the team implemented additional functions for each entity and character to update their animation states.

To render objects on the game screen, the project utilizes data types from the SFML library, specifically `sf::Texture` and `sf::Sprite`. In this setup, `sf::Texture` loads the entire sprite sheet into VRAM exactly once, while `sf::Sprite` is used to display the rendered visual on screen—it contains no image data itself and merely references the `sf::Texture`.

Before rendering a sprite sheet, we define the width and height (typically a square) for each frame in pixels so the resource manager can segment the sprite sheet into individual frames. This allows the state update function to easily track the active frame index and determine which frame sequence to use when a character changes direction or when vehicles and animals move.

Next, we use `sf::IntRect` to crop each frame from the sprite sheet. `sf::IntRect` is extremely convenient for switching entity and character animations: by simply specifying the top-left corner coordinates along with the frame's width and height, we can transition animations via the `setTextureRect` function.

When combined with customized frame durations and frame traversal logic for mobile objects, animations become much smoother and more coherent compared to simply

loading sprite sheets without dedicated code. Furthermore, programming sprite sheet logic optimizes overall performance and system resources by significantly reducing draw calls.

For the player character in particular, C++ programming for the sprite sheet is even more essential. Without proper logic, relying solely on state changes via the sprite sheet would make a jump across tiles look like instant teleportation rather than visible motion. In this project, to execute jumps without triggering unwanted mid-air collisions, the character's hitbox stays at the starting tile while ascending and snaps to the target tile at the apex of the jump (start of descent), before actually landing, to avoid mid-air collisions. Prior to jumping, the character also verifies whether adjacent tiles permit movement; if the conditions are not met, the character remains stationary.

3.5.3. Ease-out quadratic interpolation

When a character jumps from an old tile to a new one, defining the jumping trajectory—specifically the character's displacement from origin to destination—is crucial. This is particularly important when a jump animation is present, as combining physical displacement with visual animation produces a significantly more realistic and dynamic effect. If this aspect is neglected, the visual experience becomes jarring, as the character may simply translate linearly between tiles without a natural jumping arc, detracting from the game's overall fluidity.

While various interpolation techniques exist, this project adopts quadratic ease-out interpolation to render the jump movement smoothly and efficiently. We chose quadratic ease-out over linear interpolation because linear interpolation maintains a constant velocity, causing the landing speed to equal the takeoff speed. This results in unnatural, abrupt motion that can be visually uncomfortable for the player. Conversely, quadratic ease-out interpolation provides maximum velocity at takeoff and decelerates smoothly to zero as the character reaches the destination.

In game development, positional interpolation is governed by an interpolation function `h(t)`, where the character's coordinates are defined as:

$$x = x_{old} + (x_{new} - x_{old}) * h(t)$$

$$y = y_{old} + (y_{new} - y_{old}) * h(t)$$

Here, `h(t)` (represented as `t_move` in the source code) denotes the interpolation function, while `x` and `y` correspond to `ground_x` and `ground_y` in the implementation. We define our interpolation function as:

$$h(t) = 1 - (1 - air_t)^2$$

where `air_t` represents the normalized airborne progress of the character within the range `[0, 1]`.

Pseudocode:

```
CLASS Easing
    FUNCTION EaseOutQuad(t)
        RETURN 1 - (1 - t)*(1 - t)
    END FUNCTION
END CLASS

CLASS Entity
    startX = 0
    endX = 0
    x = 0
    duration = 0.3
    elapsed = 0
    isMoving = FALSE

    FUNCTION MoveTo(targetX)
        startX = x
        endX = targetX
        elapsed = 0
        isMoving = TRUE
    END FUNCTION

    FUNCTION Update(delta)

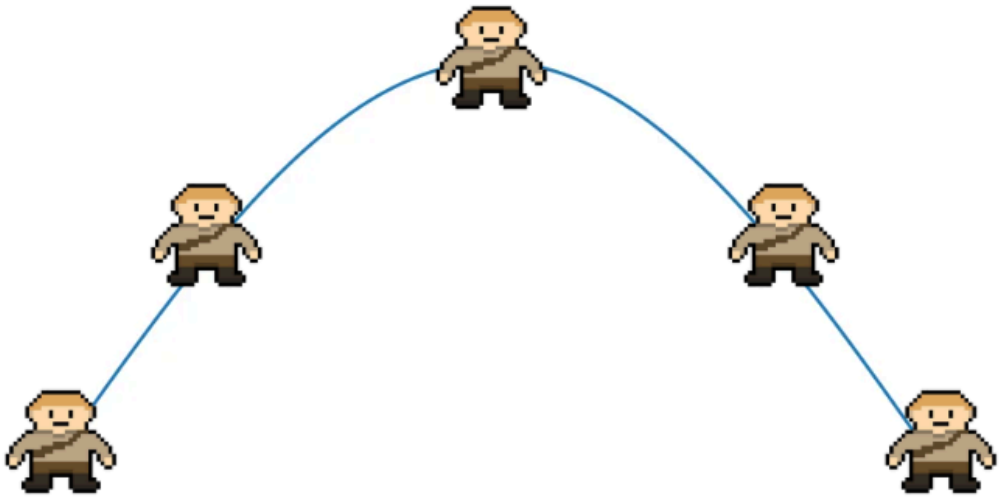
        IF NOT isMoving THEN
            RETURN
        END IF

        elapsed = elapsed + delta
        t = MIN(elapsed / duration, 1.0)
        x = startX + (endX - startX) * Easing.EaseOutQuad(t)
```

```
31
32          IF t >= 1.0 THEN
33              isMoving = FALSE
34          END IF
35      END FUNCTION
36  END CLASS
```

3.5.4. Half-sine wave arc

While interpolation governs the horizontal progression of player movement, it does not define the vertical trajectory of the jump itself. Therefore, a mathematical function must be established to model the character's vertical arc when transitioning from the origin tile to the destination tile. To address this, we adopted a half-sine wave arc function.



Half-Sine Wave Arc Trajectory

We selected a half-sine wave trajectory over a parabolic curve because parabolic arcs exhibit steep gradients at both the takeoff and landing points. This high initial and final vertical velocity often makes the jump animation appear jerky and unnatural. In contrast, the half-sine curve provides a smoother transition that allows the character model to visually wind up for takeoff and absorb the impact upon landing. Furthermore, parabolic trajectories strictly model free-fall physics, which is often ill-suited for stylized game animations that require squash-and-stretch anticipation. The gradual elevation changes of a half-sine wave yield a significantly more fluid and visually appealing motion.

To implement this trajectory, we utilized C++'s `<cmath>` library for the sin function and defined the constant value of `pi`. The maximum jump height (represented as `JUMP_ARC_HEIGHT` in the source code) was set to 65.0 pixels, and the airborne duration (air_t in the source code) was normalized to the range `[0, 1]` using `std::clamp`. Finally, the vertical displacement of the jump arc is calculated using the following formula:

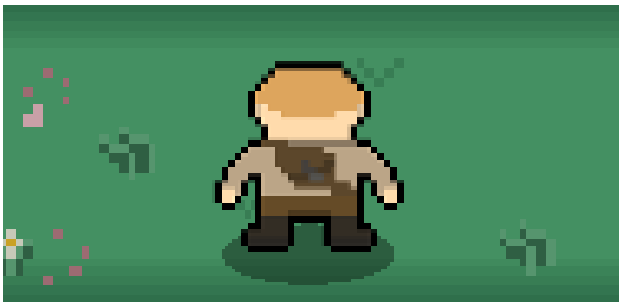
```
z_height = JUMP_ARC_HEIGHT * sin(pi * air_t)
```

Pseudocode

```
1   ALGORITHM CalculateJumpElevation(progress):
2       //inputs: progress (from 0.0 to 1.0)
3
4       CONSTANT JUMP_START = 0.15
5       CONSTANT JUMP_DURATION = 0.70
6       CONSTANT JUMP_ARC_HEIGHT = 65.0
7       CONSTANT PI = 3.141592653589793
8
9       //calculate air_t by normalizing progress and clamp to
    [0,1]
10      air_t = CLAMP((progress - JUMP_START) /
    JUMP_DURATION,0.0,1.0)
11
12      //calculate the jump height by half-sine wave arc
13      z_height = JUMP_ARC_HEIGHT * SIN(PI * air_t)
14
15      RETURN z_height
16  END ALGORITHM
```

3.5.5. Dynamic shadow & anchor compensation

To further enhance the jump effect and achieve a more realistic and seamless visual presentation, we implemented Dynamic Shadow and Anchor Compensation techniques to convey a sense of 3D depth to the player (commonly referred to as a 2.5D space).



The shadow of the character

For the Dynamic Shadow technique, we initialized the shadow using an `sf::CircleShape` and compressed its Y-axis by a factor of 0.35 to form an elliptical shape. During the jump, the shadow scales uniformly according to the formula:

```
shadow_scale = 1 / (1 + 0.008 * z_height)
```

where z_height represents the character's vertical arc elevation. Furthermore, the shadow's opacity decreases as the character gains height, calculated as:

```
alpha = max(30, 120 - z_height * 1.2)
```



The shadow of the character when they jump

The shadow's position remains strictly anchored at `ground_x`, `ground_y`, independent of `z_height`, allowing the character sprite and its shadow to visibly separate during the jump ascent.

To preserve the integrity of the hitbox system, the origin point of the sprite is locked at `(0, 0)`. Consequently, when the character compresses or stretches during animation keyframes in preparation for a jump, the feet would naturally drift away from the ground plane. To compensate for this distortion, we calculate the scaling deltas as follows:

```
delta_w = FRAME_W * (scale_x - base_scale_x)
delta_h = FRAME_H * (scale_y - base_scale_y)
```

We then determine the final rendering coordinates as:

```
render_x = ground_x - delta_w / 2

render_y = ground_y - delta_h - z_height
```

(which combines horizontal positional interpolation with the vertical jump trajectory). This ensures that the character's feet remain firmly anchored to the ground plane (`ground_y`) regardless of any scaling or deformation applied to the sprite.

4. Environment & terrain systems

4.1. Core architecture

This system is built upon a common base class, Lane, and derived classes representing 4 distinct terrain types: Grass, Safe, Road, and River.

The base class Lane (lane.h) serves as the backbone, defining the common interface and behavior for all lane types. Its main responsibilities include:

- **State Management:** Tracks lane position (`position`), game difficulty (`level`), and the general movement direction of dynamic objects (`direction`).
- **Object Management:** Stores and controls two object lists: static objects (`list_static`) and dynamic objects (`list_dynamic`). It also handles random coin generation.
- **Collision System:** Provides standard collision checking methods: obstacle collision (`checkBlockedCollision`), lethalCollision(`checkLethalCollision`), and coin pickup (`checkCoinCollision`).
- **Polymorphism:** Exposes pure virtual methods (`spawnObject`, `getType`, `seedObject`), allowing subclasses to define how their specific objects are spawned.

4.2. Terrain specifications

4.2.1. Lane grass



This is a spawning area for animals (`Animal`) moving horizontally across the screen. A remarkable characteristic of animals is their ability to stop midway without warning.

4.2.2. Lane safe



Serves as a resting area, giving players time to plan their next move. The safe lane spawns a static object (`Rock`). People can not walk into the cell that has the rock object.

4.2.3. Lane road



This is the traffic zone designated for vehicles (`Vehicle`). It features a traffic light system, which spawns the traffic light.

4.2.4. Lane river



Instead of spawning blocking obstacles, it spawns moving logs (`Wood`) that the player can step on. The player will drown when step on the river. In addition, the player standing on the logs will drift along the velocity of the logs.

5. Audio & sound system

5.1. Game audio classification

The audio system is divided into two separate processing pipelines to optimize performance:

- Background Music (BGM): Implemented using '`sf::Music`', which streams audio directly from the disk instead of loading the entire file into memory. Background music is configured to loop automatically and is paused when the player enters the menu or reaches the Game Over state.
- Sound Effects (SFX): Implemented using '`sf::SoundBuffer`' (stored in RAM) and '`sf::Sound`' (used for playback). Sound effects are categorized into three groups:
    - User Interface (UI): Provides feedback for user interactions, such as Click, Hover, and Back.
    - Gameplay: Covers player-related events, including Jump, Crash, GameOver, and HighScore.
    - Ambient: Simulates environmental sounds to create a more immersive game world, including CarHorn, TruckHorn, and Bird sound effects.

5.2. Optimization techniques

5.2.1. Singleton design pattern and enum mapping

The '`AudioSystem`' class is implemented using the Singleton design pattern ('`AudioSystem::getInstance()`'), allowing sound playback to be invoked from anywhere in the game through the '`Audio()`' macro.

Instead of passing file paths as strings, which are prone to errors, the system maps audio files using the '`enum class SFX`' and '`enum class Music`' enumerations. This approach provides greater type safety, improves code maintainability, and simplifies audio management.

5.2.2. Preloading (Resource preloading)

To minimize stuttering during gameplay, the '`AudioSystem::preloadAll()`' function is invoked when the game starts to preload all '.wav' files into RAM through the '`ResourceManager`'. As a result, sound effects can be played almost instantly when needed, with virtually no playback latency.

5.2.3. Dynamic allocation (Concurrent audio playback)

Whenever 'playSFX()' is called, the system dynamically allocates a new 'sf::Sound' object on the heap using the 'std::unique_ptr<sf::Sound>' smart pointer and stores it in the 'active_sounds_' list. This approach enables the game to play dozens of overlapping sound effects simultaneously without interrupting sounds that are already playing.

5.2.4. Automatic memory cleanup (Garbage collection)

To prevent memory leaks caused by continuous dynamic allocation, the 'AudioSystem::update()' function is called once every frame. This function combines 'std::remove_if' with a lambda expression to scan the 'active_sounds_' list and remove 'sf::Sound' objects whose playback status is 'Stopped'. The corresponding memory is then released automatically, ensuring efficient memory management throughout gameplay.

5.2.5. Random ambient sound handling

To prevent audio clutter when a large number of vehicles are present, the 'Vehicle' and 'Animal' classes use a timer ('sound_timer_') together with a randomized cooldown interval ranging from 15 to 35 seconds. Each entity emits its sound only after the cooldown has elapsed, creating a more dynamic and immersive ambient soundscape without overwhelming the player with excessive noise.

6. Data persistence

6.1. Save/load game processing

6.1.1. Introduction to binary file persistence

Data persistence is a critical component of game development, ensuring that a player's effort and progress are preserved across multiple gameplay sessions. In this project, a binary file system was chosen as the primary mechanism for storing and retrieving game data. Unlike plain text formats, which convert numerical values into readable characters and introduce formatting overhead, binary files store data as a direct, unedited sequence of bytes identical to how it exists in the system's memory.

This approach provides significant advantages in terms of read and write performance, file size optimization, and a basic layer of data obfuscation against casual tampering. Because the saved file is not in a human - readable text format, it prevents users from easily opening and modifying their score or position using a standard text editor. This provides a simple but effective way to discourage basic cheating and preserve game integrity.

Implementing this system involves two distinct operational layers: organizing the high-level workflow of game state management, and executing the low-level conversion of object data.

6.1.2. Save and load game procedure

In our project, the save and load system is controlled by a main management class, which coordinates when to save or load the game state. When the player chooses to save, this manager pauses the gameplay and collects the current information from all active objects, such as the player’s position, current score, and the locations of moving obstacles. When loading, the manager resets the current game screen and recreates these objects using the previously saved data.

To follow good object-oriented programming design, the main manager does not directly modify or access the private variables of other objects. Instead, it calls specific functions inside each class, letting the player and obstacles handle their own data. This workflow focuses on gathering and distributing the necessary game variables. However, to actually store these running variables into a permanent file on the hard drive, the game must convert the memory data into a readable format. This transition leads directly to the next steps of binary serialization and deserialization.

6.2. Binary file serialization & deserialization

Serialization is the process of taking the active variables of game objects – such as integers for scores or floats for positions – and converting them into a raw sequence of bytes. Using standard C++ file streams like `std::ofstream` with the `std::ios::binary` flag, these bytes are written directly to a file.

Conversely, deserialization is the exact reverse process used when loading a game. The system opens the binary file using `std::ifstream` and reads the byte stream back into the computer's memory to rebuild the exact state of the game objects.

A major challenge in this phase is handling pointers and dynamic memory. Since memory addresses change every time the game restarts, saving raw pointer addresses would cause the game to crash upon loading. To prevent this, our serialization logic is designed to extract and save the actual data values inside those pointers rather than the addresses themselves. By using these two processes, the game can safely and accurately restore the player's exact position and progress.

6.3. Binary file encryption

6.3.1. Overview of binary file encryption

While binary files inherently hide data behind raw byte streams - making them far less accessible than plain text - they remain vulnerable to basic reverse-engineering and hex-editing tools. Dedicated players can easily inspect binary patterns, locate numerical variables like scores or player coordinates, and alter them to cheat. Implementing binary file encryption adds a necessary security layer that actively scrambles the byte stream. This ensures that even if the raw binary file is extracted, its contents remain completely unreadable and tamper-proof without the proper decryption mechanism.

6.3.2. Symmetric XOR Cipher technique

To secure saved data efficiently, the system uses a symmetric XOR cipher paired with a static secret key. Symmetric encryption relies on the exact same key for both encoding the saved file and decoding it upon loading. During serialization, as the game state is converted into bytes, each byte undergoes a bitwise XOR **($\oplus$)** operation with a corresponding byte from the secret key, transforming clear game data into obfuscated output.

Because the XOR operation is mathematically self-reversing - as $(A \oplus K) \oplus K = A$ - applying the identical secret key to the encrypted stream during deserialization perfectly restores the original binary data. This technique offers robust defense against casual tampering while maintaining virtually zero processing overhead during game save and load transitions.

VI. CONCLUSION

The Crossy Road project successfully demonstrates the design and implementation of a fully featured 2D arcade game built from scratch using Modern C++ (C++17) and the SFML 3.1.0 multimedia library. Throughout the development lifecycle, the project strictly adhered to clean software engineering practices, robust object-oriented paradigms, and system-level performance optimizations.

1. Project achievements & functional delivery

Complete Core Gameplay: Implemented procedural endless map generation, AABB and predictive collision detection, fluid grid-based character movement with Linear Interpolation (Lerp), safe-distance steering behaviors for traffic, and an auto-scroll camera tracking system.

Data Persistence & Security: Engineered an efficient binary serialization mechanism utilizing Symmetric XOR Cipher encryption, ensuring sub-10 KB save slots and robust defense against external save-file tampering.

Fixed Window Architecture: Locked the logical viewport to a 1400x1050 (4:3) resolution, guaranteeing gameplay fairness (preventing FOV manipulation) and visual integrity across diverse screen displays.

2. Software architecture & OOP application

Object-Oriented Paradigms: Effectively applied Encapsulation, Inheritance, Polymorphism, and Abstraction to model a highly modular game domain (Entities, States, and Terrain systems).

Design Patterns in Practice:

- State Pattern: Governs clean, decoupled state transitions across the application lifecycle (HomeState, PlayingState, ShopState, PauseState, etc.).
- Singleton & Flyweight Patterns: Centralized resource caching in ResourceManager and AudioSystem to ensure heavy assets are loaded only once.
- Simple Factory Pattern: Dynamically encapsulates the complex instantiation logic of randomized vehicles, animals, and static objects within the Lane class.
- Polling Pattern: Continuously evaluates entity proximity to execute real-time speed adjustments and traffic light deceleration.

3. Stability, memory management & profiling

Empirical Stability: Real-time profiling via Visual Studio 2022 Diagnostic Tools validated a stable process memory footprint (~360 - 380 MB Working Set) with zero memory leaks over extended play sessions.

Advanced Memory Management: Extensive utilization of C++17 smart pointers (std::unique_ptr), STL containers (std::deque, std::unordered_map), and dynamic lane culling (Map::popLane()) guaranteed optimal heap utilization and eliminated dangling pointers.

Three-Tier Exception Architecture: Formulated a domain-specific GameException hierarchy providing strong fault tolerance, automated graceful degradation (corrupted save recovery), and a top-level application safety net.

4. Technical takeaways & team growth

Deepened practical mastery of modern C++17 memory models, low-level real-time game loops (with Delta Time synchronization), hardware-accelerated 2D graphics, and multi-threading concepts.

Fostered professional software development workflows, encompassing Git version control, GitHub collaboration, modular code reviews, and structured debugging methodologies.

VII. LIMITATIONS & FUTURE ENHANCEMENTS

1. Current system limitations

Architectural Coupling & Type Casting (Leaky Abstraction): The current Lane management class occasionally relies on runtime type identification (RTTI via dynamic_cast) to differentiate between specialized entities (such as Vehicle, Animal, and Coin) for collision avoidance and collection logic. While functional, this introduces minor architectural coupling that will benefit from a more refined polymorphism hierarchy (e.g., dedicated DynamicObject and StaticObject interfaces).

Visual Depth & Z-Ordering Artifacts: Because the engine utilizes a 2D orthographic canvas without a dedicated 3D depth-buffer, rendering priority between overlapping entities (e.g., coins residing on river logs, floating debris, and particle effects) is governed strictly by the sequential draw order of container lists rather than a centralized Z-sorting pipeline.

Static Procedural Difficulty: Obstacle generation and vehicle velocities currently scale based on fixed mathematical formulas and pseudo-random seeds. The engine lacks an adaptive Dynamic Difficulty Adjustment (DDA) system that modifies game pacing in real time according to individual player proficiency.

2. Future enhancements roadmap

Integration of Postponed Gameplay Modules:

- Power-Ups & Buffs System: Implement collectible temporary gameplay modifiers across lanes, such as Speed Boots (reduced movement cooldowns), Invincibility Shields (1-second obstacle immunity), and Hourglasses (temporary camera deceleration).
- Environmental Debuffs & Traps: Introduce specialized hazard zones, including muddy terrain (speed penalties), ice patches (directional sliding), and localized fog/smoke (reduced sightlines).

Advanced Graphics & Shaders (GLSL):

- Day/Night Cycle: Utilize GLSL pixel shaders to create smooth, ambient lighting transitions between day, dusk, and night, adjusting player visibility dynamically.
- Particle VFX Engine: Build an expanded GPU-accelerated particle system to simulate dynamic weather effects (rain, snow ripples), vehicle exhaust smoke, and liquid splash ripples upon river drownings.

Community Level Editor & Modding Support:

- Construct an integrated graphical Map Editor allowing players to design custom lane sequences, configure custom obstacle speeds, and export/share stage configurations via custom binary map files (.map).

VIII. REFERENCES

[1] Trần Đ. T., Đinh B. T., and Nguyễn T. T. M. K., *Phương pháp lập trình hướng đối tượng*. Tp. Hồ Chí Minh: Nhà xuất bản Khoa học và Kỹ thuật, 2010.

[2] S. B. Lippman, *C++ primer*, 5th ed. Upper Saddle River, N.J: Addison-Wesley, 2013.

[3] S. Meyers, *Effective C++: 55 specific ways to improve your programs and designs*, Third edition, Seventeenth printing. in Addison-Wesley professional computing series. Upper Saddle River, NJ Boston Indianapolis: Addison-Wesley, 2017.

[4] B. Stroustrup, *The C++ programming language: C++ 11*, 4. ed., 4. print. Upper Saddle River, NJ: Addison-Wesley, 2015.

[5] E. Freeman, E. Robson, K. Sierra, and B. Bates, *Head first design patterns: building extensible & maintainable object-oriented software*, 2nd edition. Beijing Boston Farnham Sebastopol Tokyo: O'Reilly®, 2021.

[6] E. Gamma, Ed., *Design patterns: elements of reusable object-oriented software*, 39. printing. in Addison-Wesley professional computing series. Boston, Mass. Munich: Addison-Wesley, 2011.

[7] S. Prata, *C++ primer plus: new C++11 coverage*, Sixth edition. in Developer's library. Upper Saddle River, NJ Toronto London: Addison-Wesley, 2012.

[8] P. Mileff, "Game loop: the heart of the game engine," *Prod. Syst. Inf. Eng.*, vol. 11, no. 3, pp. 56–68, 2023, doi: 10.32968/psaie.2023.3.4.

[9] D. Coleman, "Understanding Delta Time," Medium. Accessed: Jun. 25, 2026. [Online]. Available: https://drewcampbell92.medium.com/understanding-delta-time-b53bf4781a03

IX. ACKNOWLEDGMENTS

We would like to express our sincere gratitude to Dr. Trương Toàn Thịnh for his invaluable guidance, support, and clear direction throughout this project.

We acknowledge the support of Generative AI tools in facilitating both the development and documentation of this project:

- ChatGPT (OpenAI) was employed during the preliminary research phase to suggest relevant technical literature, game design patterns, and documentation on core gameplay algorithms.
- Claude (Anthropic) assisted the engineering phase by reviewing game script logic, identifying potential edge-case errors, and providing recommendations for debugging and optimizing system flow.
- Gemini (Google) was utilized during the writing phase to translate draft sections from Vietnamese into English, ensuring consistent technical terminology and improving the academic tone of the report.

All AI-assisted outputs served strictly as baseline suggestions. The authors manually cross-checked all references against original sources, refactored and playtested the source code within the game engine, and carefully edited the final manuscript. The authors assume full responsibility for the integrity, correctness, and originality of the entire project.